VROSTBridge

Full project specification

An open-source remote PCVR bridge
VR Rendering OutSide The Box – Bridge
Windows rendering host **1.0.0.12**
Bazzite headset client **1.0.0.12**
Windows headset client **1.0.0.12**
MIT licence, by JagerBombastic

Current handoff
Complete architecture, requirements, implementation boundaries, build and install contracts, and release acceptance, revised for 1.0.0.12 (from 1.0.0.11's first session on hardware: the automatic bitrate back to where the session started once the headset PC's GPU clock is held higher, frames that never arrived not counted against it, the GPU clock the last session needed held from the start, and the add-on warning limited to add-ons from outside SteamVR), 1.0.0.11 (a lighter picture on the headset PC, so a headset's full panel resolution fits lighter headset PCs: the frame pass with about a third less work per pixel and one shader for every GPU, the pixels the lenses never show skipped, each eye shown a step smaller rather than at half the frame rate when the graphics fall behind, the pass measured on the GPU, and the Beyond's 2560 size called Native), 1.0.0.10 (from the 1.0.0.9 test: the menu button forwarded as one clean press with the other controls kept through a short loss of focus; Game Mode's SteamVR started without gamescope's display marker and given the headset display under a plain name, a failure to take it explained at once, and the last Game Mode start kept for reports; a bitrate step for uneven arrival undone when it did not help; three tries for a base station, on Bazzite with a search before each retry and its signal), 1.0.0.9 (clean game audio: a player of VROSTBridge's own on both clients and the host's capture read straight from Windows; lost video and audio pieces rebuilt from parity; Baballonia yielding a busy processor; base stations one at a time; the 3 newest client reports kept on the host), 1.0.0.8 (steadier streams: automatic bitrate held to what the picture needs and changed on the host while the stream runs, pacing below the loss-free rate, the Bazzite headset PC balanced during sessions, base stations on at Start and off at closing, the gaze centred per calibration, and recommended settings in one step), 1.0.0.7 (a Windows client for the headset PC, one version for every package) and the releases before it, which section 2 summarises.

Validation boundary

Index operation is confirmed by the maintainer, and so are the Beyond 2e picture and tracking through the bridge (RC4.5 test); the 1.0.0.2 test confirmed Baballonia's calibration over the stream and the eye-tracked foveated stream in Desktop Mode; the RC1.3 projection checks remain on the acceptance list. The 1.0.0.4 test confirmed true blacks with 10-bit colour off and the High size (now Native) looking and tracking reasonably well; the 1.0.0.5 test a more responsive sharp region (the eye cameras at 90 pictures a second) and black blacks with the 8-bit fallback; the 1.0.0.8 test much steadier motion and picture; the 1.0.0.9 test clean game audio and, over Wi-Fi, every frame that would have been lost rebuilt from parity; the first session of 1.0.0.11 Native with eye-tracked foveation at the headset's refresh and the frame pass at 1.9 ms of GPU time a frame. Other RC2 to 1.0.0.12 features (including the network, power and wake integration, and the RC4.1 to 1.0.0.12 changes) are contract-tested only; the RC4.5 eye tracking was also run end to end in a sandbox with the real Baballonia. The Windows client (1.0.0.7) is built and tested in code only (unit tests, the native contract tests under Wine with a software Vulkan driver, and the installer, companion and page under Wine): it has not run on Windows hardware. Game Mode, calibrated gaze and the Windows client have explicit remaining gates.

Contents

1. Purpose and scope 5

2. Release baseline and evidence 7

3. Product requirements 12

4. System architecture 14

5. Session lifecycle and ownership 18

6. Optical geometry and display negotiation 21

7. Video quality and frame delivery 21

8. Tracking, input, menus and play area 32

9. Audio and optional eye tracking 33

10. Network and trust model 38

11. User interfaces and Game Mode 40

12. Installation, upgrades and stored state 42

13. Diagnostics and support 44

14. Build, dependencies and source handoff 45

15. Attribution and publication readiness 46

16. Validation and release acceptance 47

17. Priorities and future work 53

Canonical text: docs/PROJECT_SPEC.md in the source archive. The PDF presents that specification with a cover and navigation; dated release reports remain in docs/evidence/.

Documentation revision: **2026-10-13 / 1.0.0.12**. Runtime baseline: **VROSTBridge Windows host, Bazzite client and Windows client 1.0.0.12** (see 1.2 for the name; 1.0.0.12 is 1.0.0.11 with the follow-ups from its first session on hardware: the automatic bitrate back to where the session started when the headset PC's GPU clock is held higher, and seconds in which frames never arrived no longer counted against it (section 7.3), the GPU clock the last session needed held from the next session's start on Bazzite, and the host's add-on warning limited to add-ons registered from outside SteamVR, the native programs 1.0.0.11's with the new version label and the host's driver unchanged; 1.0.0.11 is 1.0.0.10 with a lighter picture on the headset PC, so a headset's full panel resolution fits lighter headset PCs: the one-pass frame shader with its foveation terms worked out once per frame and one shader for every GPU, the pixels the lenses never show skipped, each eye shown a step smaller rather than at half the frame rate while the headset PC's graphics fall behind, the pass and its plane copy timed on the GPU, and the Beyond's 2560 size called Native (section 7.3), the host's driver unchanged; 1.0.0.10 is 1.0.0.9 with the follow-ups from its test: the menu button forwarded as one clean press, with the other controls kept through a short loss of focus (section 8), SteamVR started in Game Mode without gamescope's display marker and given the headset display under a plain name, a compositor that cannot take that display stopping Start at once, and the last Game Mode start kept for reports (section 11), a bitrate step taken for uneven arrival undone when it did not help (section 7.3), and three tries for a base station (on Bazzite with a search before each retry and the signal of one not reached), the host's driver unchanged; 1.0.0.9 is 1.0.0.8 with clean game audio and lost packets rebuilt on the way: a game-audio player of VROSTBridge's own on both clients and the host's game-audio capture read straight from Windows' loopback stream (section 9), parity for video and game audio (section 7.3), Baballonia at a quarter of the normal CPU weight on Bazzite, a second try for a base station that did not answer (and on Bazzite one station at a time), and only the 3 newest client reports kept on the host, with a new driver (patches 0001-0023); 1.0.0.8 is 1.0.0.7 with steadier streams: automatic bitrate held to what the picture needs and changed on the host while the stream runs when decoding falls behind or frames go missing, video paced at 70 % of the network test's loss-free rate, the Bazzite headset PC's power limit and GPU clock balanced during sessions through steamos-manager, SteamVR 2.0 base stations turned on at Start and off when the client closes, the gaze centred per calibration with the previous calibration one step away, and Use recommended settings (section 7.3); 1.0.0.7 is 1.0.0.6 with a Windows client: the headset PC may run Windows 10 or 11 (x64) as well as Bazzite, with the same page, session flow, stream and eye tracking, D3D11VA decoding on SteamVR's GPU copied into the same frame pass, a tray icon, sign-in start, firewall rules, Wi-Fi and power handled for the session, and a per-user installer (section 4.4); also the one-pass frame shader's uniform-buffer variant for GPUs with only 128 bytes of push constants, on both clients, and one version for every package, the host otherwise unchanged; 1.0.0.6 is 1.0.0.5 with the follow-ups from its test: how far the gaze reached measured and shown with calibration guidance, the host's 10-bit conversion tried as plain SDR first, a network test that tells the host's own sending time from the network's, and the stream's controller prediction measured; 1.0.0.5 is 1.0.0.4 with the fixes from its test: exact 10-bit colour checked by the host on the AMD path, frames without a head pose counted and Headset not tracked on the client, eye cameras passed on as sent at their fastest rate, a quicker Baballonia filter, a comfort zone that keeps the sharp region still for small eye movements, a stream larger than SteamVR's size shown at that size, and a network test that measures the network and is repeated when it finds no clean rate; 1.0.0.4 is 1.0.0.3 with the fixes from its test: the sharp region held where the eyes last looked through pauses in the gaze, a processor held back by the shared power limit noticed and explained with a one-step GPU clock fix, wired pacing kept to the rate the network path carried without loss, the last frame held for up to a second before the grey view, Game Mode's D-pad and stick, and the whole session in reports; 1.0.0.3 is 1.0.0.2 with the fixes from its test: Game Mode's display reservation check, the host's SteamVR ownership, the foveated reduced area filtered and smoothed, faster eye tracking and its view, Baballonia across updates). Licence: MIT (VROSTBridge's own code; `LICENSE`, `NOTICE`, `THIRD_PARTY.md`). Status: first release; hardware support is limited to the combinations in HARDWARE_ACCEPTANCE.md, the features this specification marks as contract-tested have not been accepted on hardware, and the Windows client has not yet run on Windows hardware at all. See RELEASE_1.0.0.11.md, RELEASE_1.0.0.10.md, RELEASE_1.0.0.9.md, RELEASE_1.0.0.8.md, RELEASE_1.0.0.7.md, RELEASE_1.0.0.6.md, RELEASE_1.0.0.5.md,

RELEASE_1.0.0.4.md, RELEASE_1.0.0.3.md, RELEASE_1.0.0.2.md, RELEASE_1.0.0.1.md, RELEASE_1.0.0.md, RELEASE_1.0.0-rc4.5.md, RELEASE_1.0.0-rc4.4.md, RELEASE_1.0.0-rc4.3.md, RELEASE_1.0.0-rc4.2.md, RELEASE_1.0.0-rc4.1.md, RELEASE_1.0.0-rc4.md and RELEASE_1.0.0-rc3.md for recent changes.

1. Purpose and scope

VROSTBridge lets a powerful Windows PC render SteamVR applications for a physical DisplayPort headset attached to a separate headset PC, the client, on the same trusted home network: a Bazzite Linux PC or, from 1.0.0.7, a Windows 10 or 11 PC. The client handles local headset display, Lighthouse tracking, controller input, video decoding and audio playback. The host handles the game, virtual SteamVR devices and video encoding. The aim is a convenient, low-latency VR session started from the client, with stable image quality and correct headset geometry.

The supported product scope is Valve Index and Bigscreen Beyond-family headsets, including the Beyond 2e eye-tracking integration, with Index controllers and Lighthouse base stations. The physical headset's local SteamVR driver remains responsible for device support. Beyond model identity is taken from that driver; a family match does not prove every model, refresh rate or adapter has been qualified.

The design is general rather than tied to one handheld or dock. A Ryzen handheld with integrated Radeon graphics, running Bazzite on Wi-Fi 7 or Ethernet, is the reference test configuration. The Windows client has no reference configuration yet: it is built and tested in code (unit tests, and the native contract tests and a smoke test of the companion under Wine), not on Windows hardware. The Windows rendering PC, client GPU, dock, cable, refresh rate and network must each support the selected workload. Ethernet and Wi-Fi are both supported network paths. A functioning local headset setup is assumed before remote streaming is diagnosed.

The maintainer has confirmed Index operation both locally and through the bridge, and in the RC4.5 test reported the Beyond 2e's picture and tracking through the bridge as good; in the 1.0.0.3 test the tracking felt good from moment to moment; in the 1.0.0.4 test blacks were black with 10-bit colour off, and the High size (now Native) looked and tracked reasonably well; in the 1.0.0.9 test game audio came through clean and, over Wi-Fi, packet repair rebuilt every frame that would otherwise have been lost. The RC1.3 projection checks for the Beyond remain on the acceptance list. This specification distinguishes implemented software from hardware acceptance throughout.

1.1 Boundaries

- Windows is the rendering host. The headset client runs on Bazzite x86-64 or, from 1.0.0.7, on Windows 10 or 11 x64: one source, two builds (section 4.4). Game Mode, the headset display lease and the eye-camera helper stay Bazzite's.
- The Windows client of 1.0.0.7 is the complete client (`VROSTBridge Client.exe` and the native `VROSTBridgeClient.exe`), not the early Windows diagnostic: the CMake target that built the historical root `VROSTBridge.exe` is gone, and `scripts/Build-Windows.cmd` now builds and runs the contract tests with Visual Studio's CMake. The Windows client is built and contract-tested only; it has not run on Windows hardware.
- The product targets a trusted private LAN, not Internet streaming, hostile networks or multi-tenant hosting.
- One client owns one active host session. Multi-headset rendering is outside this baseline.
- Steam, SteamVR, physical headset drivers and graphics drivers remain separate prerequisites. Gaze inference (Baballonia) is a separate program under its own licence that VROSTBridge downloads, configures and runs only when asked (RC4.5).

- In-session changes of stream size or codec, HDR, host OpenXR gaze support and dynamic foveated rendering (games rendering fewer pixels) are not implemented product capabilities. RC4 adds AV1 as an option, a measured starting bitrate and foveated encoding, fixed or driven by Beyond 2e gaze; from 1.0.0.8 the bitrate follows the stream during a session (section 7.3).

1.2 Name (1.0.0, 1.0.0.1)

The project is **VROSTBridge** (An open-source remote PCVR bridge; programs **VROSTBridge Host** and **VROSTBridge Client**); the name stands for VR Rendering OutSide The Box - Bridge. Its logo (from 1.0.0) is in `brand/`. From 1.0.0.1 every identifier carries the name too: the state and install folders (`%LOCALAPPDATA%\VROSTBridge`, `~/.local/share/vrostbridge`, `~/.local/state/VROSTBridge`), the Windows uninstall key and Run value (`VROSTBridgeHost`), firewall rules (`VROSTBridge.*`) and QoS policy (`VROSTBridgeVideo`), the Linux programs (`VROSTBridge-Client`, `VROSTBridgeClient`, `VROSTBridgeLease`, `VROSTBridgeRuntimeProbe`, `libvrostbridge_transport.so`, `libvrosttext.so`) and desktop entries (`io.vrostbridge.*`), the session and native protocol names (`vrostbridge-session-2`, `vrostbridge-native-1`) and headers (`X-VROSTBridge-*`), the driver's files (`vrostbridge-stream.json` and the others), its log lines (`VROST_*`), the emulated headset serial, environment variables (`VROSTBRIDGE_*`) and the code (`vrost::`). 1.0.0.1 is therefore a fresh installation: it does not take over a test build's state, and both PCs need it. `tests/test_vrostbridge_1001.py` checks that the names the companion, the client, the transport and the driver share agree. The Windows client of 1.0.0.7 follows the same rule: `VROSTBridge Client.exe`, `VROSTBridgeClient.exe`, `VROSTBridgeRuntimeProbe.exe` and `vrostbridge_transport.dll`, the folder `%LOCALAPPDATA%\VROSTBridge\Client`, the uninstall key and Run value `VROSTBridgeClient` and the firewall rules `VROSTBridge.Client.*`.

2. Release baseline and evidence

Area	Current state	Evidence boundary
Windows host 1.0.0.12	1.0.0.12: the add-on warning lists only add-ons registered from outside SteamVR; the driver is 1.0.0.9's. 1.0.0.11 and 1.0.0.10: the version only. 1.0.0.9: the driver rebuilt from patches 0001-0023 (0021 parity for video and game audio, sent when the companion asks for it per session; 0022 the clients' game-audio player, which the host does not use; 0023 game audio captured from Windows' loopback stream directly, ALVR's capture as the fallback); the companion turns parity on for a driver whose manifest lists 0021 and keeps the 3 newest client reports. 1.0.0.8: the companion offers `bitrate-control` and sets the running stream's bitrate when the client's heartbeat asks for a new one (ALVR's own `session/values` interface, at most every 2 s, only while the lease's stream runs); the driver is the same build as 1.0.0.6's. 1.0.0.7: the version only. The driver rebuilt from patches 0001-0020 (1.0.0.6: 0020 asks the AMD converter for plain SDR first (the same transfer and primaries in and out, no tone mapping), checks each try with the test picture and logs it (`VROST_COLOR_PROBE`, `variant=` on `VROST_COLOR`), and adds to each `VROST_GAZE_FOVEATION` line how far the gaze and the left eye's region reached in that window (`gaze_yaw`, `gaze_pitch`, `region_x`, `region_y`); 1.0.0.5: 0018 converts to Y′CbCr on the AMD path itself with a stated matrix and range, checks it with a test picture at every stream start and labels the stream with what it measured, or streams 8-bit, and counts frames it could not send without a head pose; 0019 keeps the gaze-driven region still inside a comfort zone; 1.0.0.4: 0017 keeps the sharp region where the eyes last looked for `gaze_hold_ms` through a pause in the gaze and counts held frames and pauses; 1.0.0.3: 0016 filters the foveated reduced area from a mip-mapped, anisotropically sampled copy; 1.0.0.1 and 1.0.0.2: 0001-0015 with the VROSTBridge names; RC4.2 to 1.0.0 reused the RC4.1 driver) and the companion (1.0.0.5: `gaze_comfort` in the stream options for a driver with patch 0019, the driver's `VROST_COLOR` and `VROST_NO_HEAD_POSE` lines read into the session's telemetry (shared with the client) and reports, and Waiting for the headset’s position on the host page; 1.0.0.4: `gaze_hold_ms` in the stream options for a driver with patch 0017, held frames counted as following the eyes; 1.0.0.3: a new vrserver of the session's SteamVR joins the session, so Stop closes it; no interrupted message while SteamVR starts, a slow-start message; the previous start's SteamVR log in reports; 1.0.0.2: the version only; 1.0.0.1: one name throughout, the line under the name; 1.0.0: a rebuilt launcher carrying the icon and version resource, one page tab, licence and notices; RC4.5: the `session-gaze` feature, eye tracking turned on for a client's session; RC4.4: version and shared pages only; RC4.3: one-step headset microphone setup with VB-CABLE downloaded on request, stream recovery counted within the session; RC4.2: report upload and storage, filtered driver log in reports, reorganised page); driver built from patches 0001-0020 without FFmpeg: frame-timing lock, video pacing, gaze/optical-centre foveated encoding, AMF intra refresh, AMF output drain (0013), prediction switch (0014), startup display configuration (0015), the filtered reduced area (0016), the gaze hold (0017), exact colour and pose drops (0018), the gaze comfort zone (0019), the colour variants and	User reported successful Index streaming on RC1 and smooth RC4.1 motion over Wi-Fi 7; RC3-RC4.1, 1.0.0.1, 1.0.0.3, 1.0.0.4 and 1.0.0.5 drivers cross-built and verified offline (exports, imports, manifest); from 1.0.0.4 the driver's VROSTBridge Rust modules also run their own tests on Linux, and from 1.0.0.5 the colour rules (`host_color_tests`); physical RC4.2-1.0.0.5 acceptance pending, and the 1.0.0.6 driver (unchanged in 1.0.0.7 and 1.0.0.8) has not met hardware yet; the 1.0.0.8 bitrate change is tested against a stand-in for ALVR's interface

Area	Current state	Evidence boundary
	gaze reach (`0020-host-colour-variants-and-gaze-reach.patch`); companion stream options file, client-address reconnection, stream recovery, driver log in reports, network test sender, keep-awake, sleep after session, Wake-on-LAN readiness, optional QoS policy, quick resume	

Area	Current state	Evidence boundary
Bazzite client 1.0.0.12	Native decoder/presenter with foveated-encoding inverse, P010 10-bit, AV1, chroma siting, dithering, rotation-only frame re-aim and SteamVR-style in-headset cards; single-step motion prediction (current poses on a grid locked to the display, host extrapolates); 1.0.0.12 the bitrate's fresh start after a raised GPU clock, frames that never arrived not counted against the bitrate or the GPU clock, and the GPU clock the last session needed held from the start (the native programs 1.0.0.11's with the new label); 1.0.0.11 the frame pass with its foveation terms worked out per frame, each eye a layer of workgroups and every constant in a uniform buffer (one shader for every GPU), each output row limited to what SteamVR's visible-area mesh and lens distortion show (Skip what the lenses never show), each eye shown at 90 % or 80 % of its size while the GPU holds the stream below refresh (Keep the frame rate when this PC falls behind), GPU time stamps around the pass and the plane copy, and the Beyond's 2560 size called Native; 1.0.0.10 the system button forwarded as one press of at least 0.12 s, read also from SteamVR's own button events, with the other controls kept through a loss of focus under 0.5 s, Game Mode's SteamVR started without `GAMESCOPE_WAYLAND_DISPLAY` and given the headset display as `vrostbridge-headset-<pid>` in the runtime directory, a compositor that cannot lease it stopping Start at once, the last Game Mode start kept in `game-mode-last-start.json`, a bitrate step for uneven arrival undone when arrival is no more even, and three tries for a base station, each retry after a fresh search, with the signal heard from one not reached; 1.0.0.9 the game-audio player (patch 0022) with its Game audio line, frames and audio pieces rebuilt from the host's parity (patch 0021) and counted, Baballonia at a quarter of the normal CPU weight, base stations one at a time with a second try; 1.0.0.8 the picture's bitrate ceiling and the bitrate governor with live changes on capable hosts, wired pacing at 70 % of the loss-free rate, the performance balance through steamos-manager during sessions, base stations on at Start and off at closing over BlueZ, gaze centring per calibration and the previous calibration, Use recommended settings, and the three live switches on the page and in Game Mode; 1.0.0.7 the fused frame pass's uniform-buffer variant (`fused-uniform.spv`, until 1.0.0.10) for a GPU that offers only 128 bytes of push constants, in place of the two passes, long messages wrapping on the page, and the version (the same source also builds the Windows client); 1.0.0.6 how far the gaze reached each session (Gaze reach on the page and in Game Mode, Measure again, reports; whole degrees only) with calibration steps that say to turn the head far enough each way, the network test judging lateness by what the network added to each frame (the sender's own time measured apart, `network_spread_p95_ms`) and testing again once after a result limited that way, and the controller prediction check	Native (including software-Vulkan checks of the shaders and, from 1.0.0.11, of the client's frame path with SteamVR stood in for), Python, UI and package checks recorded; no headset attached to the build environment

Area	Current state	Evidence boundary
	(the host's extrapolation repeated on the client's own samples: `INPUT` lines, the timeline, `controller_prediction` in reports); 1.0.0.5 Headset not tracked (page, Game Mode, headset card) with head positions sent and missing counted, the stream's colour shown and logged, a stream larger than SteamVR's recommended size shown at that size, the camera helper passing pictures on as the cameras sent them at their fastest rate with damaged pictures skipped and STATS lines, Baballonia's filter at 2 Hz and 60, a network test timed by the kernel with the receiver's own drops apart and a warm-up step, repeated when it finds no clean rate; 1.0.0.4 a processor held below its own minimum clock noticed with its cause and a one-step GPU clock fix through steamos-manager, Baballonia one step lower in priority, wired pacing at or below the network test's loss-free rate (a path tested earlier is tested once more), the last frame held re-aimed for up to 1 s before the grey view, Game Mode's left and right on settings rows, repeat and one move per press, a session timeline and the gaze relay's pauses in reports; 1.0.0.3 the Game Mode reservation check that works with gamescope's file capability, the reduced area enlarged with a B-spline, the camera helper at up to 66 pictures a second, Baballonia's eye filter set once, the eye-tracking view at 20 updates a second with a trail, Baballonia kept across updates and uninstalls; 1.0.0.2 the Vulkan layer that keeps Baballonia's calibration overlay from crashing on Mesa; 1.0.0.1 one name throughout (programs, folders, desktop entries, transport library, protocol names) and the line under the name; 1.0.0 the logo (icon theme, Steam artwork, headset cards), Baballonia's calibration as a SteamVR overlay over the stream (the session keeps running), Baballonia started minimised with Show Baballonia, one page tab, the camera helper's own MJPEG server; RC4.5 eye tracking set up and run by VROSTBridge (Baballonia downloaded on request, eye cameras, relay, calibration gate, recovery); RC4.4 advice and a live readout for a held-back GPU (TDP limit or fixed clock), SteamVR compositor crash recovery, a lighter fused pass and sharper headset cards; RC4.3 fused frame pass (one pass per shown frame with contrast-adaptive sharpening), eye-tracked foveation tuning and live eye view, headset-PC graphics limit (one-tap smaller size, Auto Standard for the Beyond), Game Mode headset recognition by EDID head-mounted markers and every setting in Game Mode; RC4.2 access-point lock during sessions, Wi-Fi/Ethernet detection, Steam library entry, report upload; companion network test/automatic bitrate, measured Wi-Fi pacing, second Wi-Fi adapter pause, stream state for host recovery, live foveation status, gradual loss recovery, Wake-on-LAN, keep-awake, headset audio, reprojection check, Wi-Fi diagnostics, per-headset settings	

Area	Current state	Evidence boundary
Windows client 1.0.0.12	1.0.0.12: the bitrate's fresh start and frames that never arrived not counted against it (Windows has no GPU clock to hold). 1.0.0.11: the Bazzite client's 1.0.0.11 changes, all of them. 1.0.0.10: of the Bazzite client's 1.0.0.10 changes, the menu button, the arrival check and three tries for a base station (without the search before a retry or the signal strength). 1.0.0.9: the Bazzite client's 1.0.0.9 changes except Baballonia's CPU weight (a systemd setting; on Windows	Built and contract-tested only: Python tests with stand-ins for the Windows calls, the native contract tests cross-built and run under Wine 9.0 with Mesa's lavapipe (27 programs in 1.0.0.11, among them the client's frame path from a

Area	Current state	Evidence boundary
	Baballonia already runs one step down in priority); the player's thread in Windows' audio scheduling class; the base stations, already reached one at a time, get the second try. 1.0.0.8: the Bazzite client's 1.0.0.8 changes except the performance balance (there is no steamos-manager on Windows), with the base stations through Windows' Bluetooth API from Windows PowerShell 5.1 (`launcher/BaseStations.ps1`). 1.0.0.7: new: the same companion in the client role on the embedded CPython 3.13 taken byte for byte from the host's package, with the same page and actions (Windows wording; Game Mode, Add to Steam, the display lease, eye-camera access, Fix SteamVR reprojection and the GPU clock fix hidden), and the same native client cross-built for Windows x64 (`VROSTBridgeClient.exe`, `VROSTBridgeRuntimeProbe.exe`, `vrostbridge_transport.dll`, FFmpeg n6.1.4 DLLs, static C runtime): D3D11VA decoding on SteamVR's GPU (found by LUID), each picture copied into the plane images of the same fused pass, CPU decoding for troubleshooting, gaze over loopback UDP, stop by `quit` on stdin; a tray icon with Start and Stop session and notifications, sign-in startup, five firewall rules after one UAC prompt, Wi-Fi media streaming mode and no background scans during sessions, Best performance power mode with a receipt, SteamVR found through OpenVR's registry with a plain message when it never ran, crash records from Windows' event logs; Baballonia's pinned Windows installer run privately, or the user's own matching installation; a per-user NSIS installer and a portable ZIP	CPU-decoded frame's upload to the image SteamVR is given), installer and companion smoke runs under Wine; D3D11VA, the Windows Firewall, Wi-Fi and power calls and Baballonia's installer have not run on Windows hardware; no headset
Valve Index	Working local and bridged video, tracking and audio reported	Long-run controller recovery and broad performance matrix still need repeatable acceptance
Beyond connection	RC1.2 report confirms streaming at effective 75 Hz	Working local adapter established by user; not a blanket dock compatibility result
Beyond projection	RC1.3 corrects vertical tangent conversion and adds optical diagnostics	Code defect and regression tests confirmed; physical visual confirmation pending
Game Mode	Native SDL interface (redrawn at native resolution in RC4.2; every setting, B never exits in RC4.3; 1.0.0.4: left and right change a settings row's value, held directions repeat, one press moves once), one-button Steam entry with artwork, headset outputs ranked after the built-in screen, gamescope lease adapter recognising headsets by their EDID head-mounted markers (RC4.3); 1.0.0.10: SteamVR's environment and the adapter's socket name changed, the compositor's failure to take the display reported at once, each step of the handover logged and the last start kept for reports	Physical Game Mode display/lifecycle acceptance pending; in the 1.0.0.9 test SteamVR's compositor could not take the headset display in Game Mode (SteamVR error 496)

Area	Current state	Evidence boundary
Beyond 2e gaze	RC4.5: one switch; VROSTBridge downloads (pinned SHA-256), configures, starts and stops Baballonia 1.1.1.0rc6 with the camera helper and relay; calibration in Baballonia, once, during a session (1.0.0: as a SteamVR overlay over the stream, which keeps running; Baballonia starts minimised, Show Baballonia opens it); 1.0.0.4: the host keeps the region where	Software path run end to end in a sandbox with the real Baballonia and a stand-in camera; calibration, direction, recovery and the cameras' rates on physical hardware pending;

Area	Current state	Evidence boundary
	the eyes last looked for a second through pauses in the gaze; 1.0.0.5: the cameras' pictures passed on as sent at their fastest rate, Baballonia's filter following within two camera pictures, and a comfort zone that keeps the region still for small eye movements; 1.0.0.7: on the Windows client, Baballonia's pinned Windows installer run privately (or the user's own matching installation), the `BigEye` camera read by Baballonia itself, the processor chosen once for its eye model, gaze over loopback UDP	the Windows path tested with stand-ins for Windows, the registry, WMI and the installer only

RC1.1 improved startup diagnosis. RC1.2 reconciled a narrowly identified Beyond refresh mismatch: the runtime display property said 90 Hz while physical display timing advanced at 75 Hz. RC1.3 fixes projection conversion without changing stream layout, bitrate, pacing or host protocol. RC2 adds client-side motion prediction and the reliability/diagnostic changes in its release notes, again without changing codec, resolution, bitrate, optics or protocol identity.

Release evidence is retained under `docs/evidence/`. The RC1.3 projection record identifies native binary hashes, reused dependencies, 70 selected Python test results, native geometry/readiness checks and remaining validation. The package record covers managed upgrade, repair, retained state, launcher self-test and authenticated report access. Those tests are not substitutes for physical VR acceptance.

3. Product requirements

ID	Requirement	Baseline status
REQ-01	Start and stop both ends from the client	Implemented; recovery acceptance remains ongoing
REQ-02	Keep host idle in the tray, with Settings, Exit and status	Implemented; from 1.0.0.7 the Windows client has its own tray icon (Open VROSTBridge, Start session, Stop session, Quit)
REQ-03	Optional host sign-in startup opens neither browser nor SteamVR	Implemented; the same for the Windows client (1.0.0.7)
REQ-04	Discover and pair automatically on an eligible trusted LAN	Implemented with network policy and session leases
REQ-05	Detect physical HMD, stream dimensions, refresh and eye geometry	Implemented; Beyond projection acceptance pending
REQ-06	Forward both Index controllers, skeletal input, menus and haptics	Implemented; game and reconnect matrix incomplete
REQ-07	Keep local compositor/tracking active without a competing local Home scene	Implemented settings ownership and neutral/held remote presentation
REQ-08	Support desktop web UI and native Bazzite Game Mode controls; never use a VR headset as the Game Mode screen	Implemented (Steam entry and headset-last display ranking from RC4.2); Game Mode physical acceptance pending; on Windows (1.0.0.7) the page and the tray instead, no Game Mode
REQ-09	Transfer supported play-area geometry to host	Rectangular area and standing transform implemented
REQ-10	Preserve preferences and remove owned obsolete files during upgrades	Implemented managed install/repair/uninstall
REQ-11	Export actionable reports including retained client diagnostics on host	Implemented; from RC4.2 reports name their end, version and time and the client can send its report to the host
REQ-12	Preserve image quality while reducing latency and uneven frame delivery	Implemented bounded pipeline; further measured tuning required
REQ-13	Allow IPD/eye geometry updates during a session	Implemented geometry events and source-frame association
REQ-14	Offer opt-in Beyond 2e calibrated gaze transport, set up and run by VROSTBridge (RC4.5)	Implemented integration; sandbox end-to-end run with the real Baballonia; hardware acceptance pending
ID	Requirement	Baseline status
REQ-15	Preserve upstream attribution and review distribution obligations	Notices included; Windows host carries no FFmpeg since RC4; VROSTBridge's own code MIT-licensed (1.0.0); third-party review for public release

ID	Requirement	Baseline status
		done (1.0.0); the Windows client (1.0.0.7) carries its LGPL FFmpeg DLLs with their unmodified source and build notes, as the Bazzite client does; artwork licence not chosen
REQ-16	Spend the stream budget where the user looks (foveated encoding; gaze-driven with Beyond 2e eye tracking)	Implemented RC4; physical acceptance pending
REQ-17	Keep the host usable from another room: stays awake in sessions, can be woken, may sleep after use	Implemented RC4 (keep-awake, Wake-on-LAN, sleep after session); physical acceptance pending
REQ-18	Choose a bitrate and pacing that the measured network path carries	Implemented RC4 (network test, automatic bitrate, link-rate pacing); physical acceptance pending
REQ-19	Tell the user in the headset what is happening when video stops	Implemented RC4 in-headset status panel
REQ-20	Bring the stream back after a network interruption without user action	Implemented RC4.1 (client-address reconnection, host stream recovery); physical acceptance pending
REQ-21	Keep head-locked content steady while the head moves	Implemented RC4.2 (even pose timing, frame re-aim, alignment check); physical acceptance pending
REQ-22	Keep Wi-Fi from pausing the stream to scan for other access points	Implemented RC4.2 (session access-point lock with restore); physical acceptance pending
REQ-23	Make the headset microphone available to Windows apps without the user finding audio software	Implemented RC4.3 (host Set up headset microphone: VB-CABLE by VB-Audio downloaded on request, verified, its own setup opened); physical acceptance pending
REQ-24	Tell the user when the headset PC cannot present the stream at the headset refresh, and how to fix it	Implemented RC4.3 (graphics limit with re-aim pause, in-headset line, one-tap Standard, reprojection repair advice); RC4.4 names a held-back GPU (TDP limit or fixed clock) first and shows its clock live; physical acceptance pending
REQ-25	Run the headset client on Windows 10 or 11 (x64) as well as Bazzite, with the same page, session flow, stream, re-aim, foveation and eye tracking, and Windows' own settings handled for the session	Implemented 1.0.0.7 (section 4.4); built and contract-tested on Linux and under Wine; not yet run on Windows hardware
REQ-26	One version for every package (host, Bazzite client, Windows client), installed together	Implemented 1.0.0.7 (`tests/test_vrostbridge_1007.py`)

4. System architecture

4.1 Windows rendering host

SteamVR runs a patched ALVR virtual HMD/controller driver. Games render for that virtual headset. ALVR captures and hardware-encodes the view (optionally foveated: edges compressed before encoding, with per-frame centres) and sends video and audio to the native client. Tracking, controller and optional gaze information flow back from the client. Host device properties follow the negotiated physical headset where supported, including native Index identity/controller mappings. This improves SteamVR behaviour but does not turn the virtual device into a USB-connected physical headset or guarantee vendor firmware tools and every dashboard option work remotely.

The Python companion owns the host profile, registration, lifecycle, diagnostics and desktop UI. Per-session driver options that must not change ALVR's session schema (pacing rate, intra refresh, gaze and optical-centre foveation) are written to `vrostbridge-stream.json` beside `session.json`, which the patched driver reads at each client connection; the frame-timing lock uses `vrostbridge-timing.json` the same way. A native Windows launcher starts its bundled Python runtime. The tray uses Win32 APIs and the shared icon. A helper transfers supported chaperone geometry through OpenVR; a separate read-only audio helper checks microphone endpoint availability. A packaged host does not require a user-installed Python interpreter.

4.2 Linux headset client

The local SteamVR runtime owns the actual HMD, Lighthouse devices, local tracking prediction, chaperone and final lens/display presentation. VROSTBridge's C++ client receives host frames, decodes them using FFmpeg (VA-API; NV12 or 10-bit P010), performs GPU colour conversion through Vulkan, undoes foveated encoding per eye with the centres sent for each frame, and submits per-eye textures and the corresponding source pose to the local compositor. From RC4.2 a compute pass can first turn each frame from its source pose to the pose the compositor will display it with (section 7.3). A head-locked OpenVR overlay shows a status card in the headset when video is not fresh. A Rust C ABI adapter uses the pinned ALVR client core for streaming, tracking and audio.

Keeping SteamVR on the client is part of this implementation, even though the game renders on the host. Removing its compositor/driver stack would require a different local runtime integration. The lightweight goal is achieved by suppressing SteamVR Home and the local dashboard, bounding work and queues, and using hardware decoding. It does not imply zero local rendering or zero SteamVR overhead.

The shared Python companion supplies the desktop web interface and the native Game Mode interface, starts the runtime, negotiates the headset profile, stores preferences and collects diagnostics. `VROSTBridgeRuntimeProbe` verifies the physical runtime before requesting a host stream. `VROSTBridgeLease` bridges the gamescope lease to the local runtime when the Game Mode integration is enabled. From 1.0.0.7 the same client also builds for Windows (section 4.4).

4.3 Source ownership

Location	Responsibility
`client/platform/linux/`	Native receive/decode/present loop, Vulkan, runtime probe, DRM-lease helper; from 1.0.0.7 the loop, Vulkan, decoder, shaders and probe also build the Windows client, with Windows sections
`client/platform/windows/`	1.0.0.7: D3D11VA decoding on SteamVR's GPU (`d3d11_video.*`), the upload of each decoded picture into Vulkan (`vulkan_upload.cpp`), the command pipe, console events and scheduling (`process.hpp`), DLL loading, and the programs' version resource and manifest
`client/device/`	OpenVR tracking (including the RC4.2 pose-sampling schedule), optical geometry, input and runtime readiness; from 1.0.0.7 the plane upload shared by D3D11VA and CPU decoding (`planar_upload.hpp`, `decoded_planes.hpp`) and the gaze input over loopback UDP (`gaze.hpp`)
`client/ui/`	Text and shape rasteriser (stb_truetype), in-headset status cards, `libvrosttext.so` for the Game Mode screen
`client/transport/`	Pinned ALVR client-core adapter and C ABI
`core/`	Shared frame, geometry, presentation and input contracts
`host/`	Host profile, lifecycle, audio, telemetry and native helpers
`launcher/`	Desktop/Game Mode UI, tray, startup, session broker, reports and maintenance; from 1.0.0.7 the Windows client's network and power calls (`windows_network.py`, `windows_power.py`), firewall script (`Client-Firewall.ps1`), and Windows branches in the client's runtime, network, Wi-Fi, power, eye-tracking, tray and sign-in modules
`patches/`	Twenty-three reviewed modifications to the pinned upstream stack (0001-0006 consent, networking, controllers, capture/AMF, native protocol, chaperone; 0007 frame-timing lock; 0008-0012 stream statistics, pacing, gaze/optical-centre foveation, AMF intra refresh; 0013-0015 AMF output drain, prediction switch, startup display configuration; 0016-0020 foveation prefilter, gaze hold, exact colour and pose drops, gaze comfort, colour variants and diagnostics; 0021-0023 forward error correction, the client's audio playback, the host's game-audio capture)
`packaging/`, `scripts/`	Managed installers, builds, provenance and source archive tooling, the scripts that build or start each part from the source (`scripts/Build-*`, `scripts/Start-*`) and the CMake toolchains for cross-building (`scripts/cross/`; from 1.0.0.7 `windows-clang-cl.cmake` for the Windows client); from 1.0.0.7 the Windows client's cross-builds (`scripts/Build-FFmpeg-Windows.sh`, `Build-Client-Transport-Windows.py`, `Build-Client-Windows.py`), installer (`packaging/windows-client-installer.nsi`) and package builder (`packaging/build_windows_client.py`)
`tests/`, `docs/evidence/`	Automated checks and retained validation records

4.4 Windows headset client (1.0.0.7)

The Windows client is the same client built for Windows 10 or 11 (x64). The local SteamVR still owns the headset and the native client is its scene application: ALVR's client core behind the same C interface (`vrostbridge_transport.dll`, built from the same Rust source as Linux's `libvrostbridge_transport.so`, with no further ALVR patch), the same fused frame pass with re-aim, foveation inverse, sharpening and colour, and the same in-headset cards. The companion is the same Python program in the client role, started by `VROSTBridge Client.exe` (the host's launcher built for the client) on the embedded CPython 3.13 taken byte for byte from the host's package. It serves the same page with the same actions; its status says `platform: windows`, and the page then uses Windows wording and hides Bazzite's controls.

What the two clients share: the page and session flow, discovery and pairing, the network test and automatic bitrate, stream sizes, foveation and sharpening, eye tracking with Baballonia, reports and the session timeline, gaze reach and the controller checks. What stays Bazzite's: Game Mode and its screen, Add to Steam, the headset display reservation and DRM lease (`VROSTBridgeLease`), the eye-camera helper (go-bsb-cams) and its udev rule, Baballonia's calibration launcher and Vulkan layer (with the OpenXR runtime check), Fix SteamVR reprojection and steamos-manager's GPU clock fix. On Windows these are hidden from the page, and their actions, if called, say in plain words why they do not apply there.

Decoding. FFmpeg n6.1.4 as shared libraries for Windows (`avcodec-60.dll`, `avutil-58.dll`, `swscale-7.dll`: the H.264, HEVC and AV1 decoders with D3D11VA and the Vulkan hardware context, cross-compiled with mingw-w64 from the same unmodified source as Bazzite's, LGPL 2.1 or later). The decoder runs on the DXGI adapter whose LUID is the one SteamVR's Vulkan GPU reports, and the LUID is checked again once FFmpeg has opened the device. The adapter's D3D11 decoder profiles decide what the client offers the host: 10-bit only when its HEVC Main10 decoder outputs P010, AV1 only when it has an AV1 decoder. Each decoded picture is downloaded (`av_hwframe_transfer_data`) straight into a persistently mapped, host-visible staging buffer and copied from there into the same plane images the Bazzite client fills, which the same pass reads (`client/device/planar_upload.hpp`); the time this takes is reported where Bazzite reports its import (`map_sync_import` in the `TIMING` line). A Beyond's Standard stream (4032 × 2016 before foveation) is about 12 MB a picture each way in 8-bit, twice that in 10-bit, and under 4 MB with eye-tracked foveation. CPU decoding (`--decoder cpu` or `--software-decode`; troubleshooting only: H.264 and HEVC, no AV1, and the companion asks for no foveation) writes its planes into the same staging buffer and takes the same pass. A status line already reports whether the GPU's Vulkan driver could decode the stream itself (Vulkan Video, not used yet).

The native client's process. The companion starts it directly (Bazzite's SteamVR environment wrapper has no Windows counterpart) with its standard input as the command pipe. Stop writes `quit` and waits up to 8 seconds, then closes the pipe, which also stops it; the process is ended only as a last resort. Gaze reaches it on a UDP port on 127.0.0.1, chosen free at each Start (`--gaze-udp`), which the client binds for its exclusive use: the same `VRG1` datagrams as on Bazzite, stamped with Python 3.13's `time.monotonic_ns()`, which on Windows is the native client's own clock. SteamVR is found through OpenVR's registry, `%LOCALAPPDATA%\openvr\openvrpaths.vrpath` (its programs in the registered runtime's `bin\win64`), started with its own `vrstartup.exe`, asked to quit with `vrmonitor://quit`, and its processes are identified by exact path and creation time. SteamVR writes that file the first time it runs, so on a PC where it never has, Start says so in plain words.

Windows' own settings, for the session.

- **Tray icon:** Open VROSTBridge, Start session, Stop session and Quit. A session started from the tray, where no page may be open, reports its first picture (Streaming), or why it stopped or did not start, as a Windows notification.
- **Start when I sign in** (an installer choice, or the page's switch): the Run value `VROSTBridgeClient` in `HKCU\Software\Microsoft\Windows\CurrentVersion\Run` with `--background` starts the tray icon only, with no browser and no SteamVR.

- **Firewall:** Allow home-network connection runs `launcher/Client-Firewall.ps1` after one administrator (UAC) prompt. It creates five inbound rules named `VROSTBridge.Client.*`, Private profile and local subnet only, each for the program that listens and carrying an ownership marker: TCP 9943, UDP 9944 and UDP 5353 for `VROSTBridgeClient.exe`, and TCP 9945 and UDP 9944 and 9946 for the companion's `python.exe`, exactly the Bazzite client's ports. The check also reports a network Windows classes as Public and block rules that a declined Windows Firewall prompt left behind.
- **Wi-Fi** (Native Wifi API): during a session the adapter runs in media streaming mode with background scanning off (Stay on one access point), and other Wi-Fi adapters on the same network are disconnected and reconnected at Stop, with a receipt. Turn off Wi-Fi power saving sets the power plan's wireless setting to Maximum Performance, on battery and plugged in. Windows 11 24H2 and later name the network and its access point only to desktop apps with location permission; without it the page says so (`location_blocked`), roaming detection and the pausing of other adapters are off, and everything else works.
- **Power:** Windows' power mode Best performance for the session, with a receipt (`session-power.json`), restored at Stop or, after a session that ended in a crash, at the next start (a mode changed meanwhile is kept); the PC and its display are kept awake.
- **Reports:** SteamVR's compositor and server crashes and hangs and the native client's (Windows' Application log) and display-driver resets (Display event 4101); the client's log is `client-latest.log`, beside the host's on a PC with both; Baballonia's own log comes from `Documents\ProjectBabble`.
- **Audio:** SteamVR on Windows switches sound to the headset itself, so the client changes no audio device and names the current defaults.
- **A held-back PC:** the graphics and processor advice is judged from what Windows reports (it has no general GPU clock or power reading) and has no one-tap fix.

Eye tracking on Windows is in section 9, the installer in section 12 and the build in section 14.

Decisions (1.0.0.7)

- **One client, two builds.** The Windows client is the Linux client's source with Windows sections, not a second program, so the stream, re-aim, foveation, colour and every page action behave the same and are tested once. Only decoding, process plumbing, the camera path and Windows' own settings differ.
- **A copy first, sharing later.** On Bazzite FFmpeg's VA-API surfaces are imported into Vulkan as DMA-BUFs and copied on the GPU. FFmpeg has no such mapping from a Direct3D 11 picture into Vulkan, so each picture is downloaded into a staging buffer and copied into the same plane images. That keeps the tested frame pass unchanged, at the cost of memory bandwidth, which the timing details report. Sharing the decoder's texture with Vulkan, or decoding with Vulkan Video, is future work, to be taken up if reports show the copy matters.
- **SteamVR's GPU, by LUID.** Decoding must run on the GPU that renders for SteamVR, or every picture would cross between GPUs. FFmpeg's D3D11VA device takes an adapter index and falls back to the default adapter without saying so, so the client checks the opened device's LUID. What it offers the host (AV1, 10-bit) follows that adapter's decoder profiles, so the host never sends what this PC cannot decode.
- **Gaze over loopback UDP.** Windows has no Unix datagram sockets (its Python has no `AF_UNIX`), so the relay sends to a UDP port on 127.0.0.1, chosen free at each Start, that the native client binds for its exclusive use. The datagrams and the freshness rule are Bazzite's, and Python 3.13's monotonic clock is the native client's clock on Windows, which is one reason the companion needs 3.13 there.
- `quit` **on stdin.** Windows has no SIGTERM, and ending the process would skip the native client's clean-up (SteamVR's scene application, the transport), so the companion asks it to quit on its command pipe. Bazzite keeps SIGTERM.
- **No Game Mode on Windows.** On Bazzite, Game Mode is gamescope's session, which must reserve the headset's display and lease it to SteamVR. SteamVR on Windows drives the headset itself, and the

companion lives in the tray, so Windows does not need it: Game Mode, Add to Steam and the display lease stay Bazzite's.

- **Firewall rules no wider than Bazzite's.** The rules open exactly the Bazzite client's ports, each only for the program that listens on it, on Private networks and the local subnet, after one administrator prompt, and the uninstaller removes them: a Windows client is no more reachable than a Bazzite one.
- **Baballonia kept private, the user's own respected.** Project Babble publishes Baballonia for Windows only as an installer. VROSTBridge runs it silently into its own folder, in a job object that lets it start no other program, and puts back the uninstall entry and shortcuts it wrote, so VROSTBridge's copy stays private, as on Bazzite, and a Baballonia the user installed keeps its own entry and shortcuts. A matching installation the user already has (its version and three program files' SHA-256) is used instead of a download and never removed. Baballonia's own Windows default runs its eye model on the GPU, which the decoding and SteamVR's compositor need on time, so VROSTBridge chooses the processor once and keeps any later choice made in Baballonia.
- **No Visual C++ runtime, one Python.** The native programs link the C runtime statically and the FFmpeg libraries link libgcc statically, so the client installs no Microsoft redistributable. The companion's Python is the host package's, verified byte for byte, so both Windows packages carry one checked runtime.
- **Cross-built on Linux.** The Windows client builds with clang-cl and lld-link against an xwin-style SDK, as the host driver does, with build paths mapped out and content-derived timestamps, so a rebuild from the same inputs gives the same programs (the transport DLL when it is linked without its debugging database, `--no-pdb`) and no Windows build machine is needed. Its C++ tests run there under Wine.
- **A uniform-buffer variant rather than two passes (both clients).** Some drivers, such as some AMD drivers on Windows, offer only the 128 bytes of push constants every Vulkan driver must, less than the fused pass's 208. RC4.2's two passes would drop sharpening and resample twice, so `fused-uniform.spv` reads the 96 bytes of eye re-aim rows from a small uniform buffer (binding 3) and keeps the first 112 bytes as push constants; `VROSTBRIDGE_FUSED_UNIFORM_ROWS=1` chose it on any GPU. The shader contract test ran every case through both variants and compared the bytes (23 runs, none different, on Linux and under Wine). From 1.0.0.11 the pass takes all of its constants from a uniform buffer and no push constants, so one shader serves every driver and the variant is gone (section 7.3).
- **One version for every package.** The host's driver is unchanged (the 1.0.0.6 build, patches 0001-0020) and its companion shares the Windows client's network code (adapters read through Windows' own interface, PowerShell only as the fallback); the host and either client of a release report the same version and are installed together.

5. Session lifecycle and ownership

5.1 Idle and discovery

The host companion waits in the tray. Its Settings action opens the local web UI; Exit closes the companion through its normal cleanup path. Optional sign-in startup stays quiet and does not launch SteamVR. The host page provides configuration, status and reports; the client is the normal Start-session authority. The Windows client (1.0.0.7) waits the same way in its own tray icon, from which a session can also be started and stopped; it opens neither a browser nor SteamVR when started at sign-in.

The client discovers an eligible host on the selected private LAN. Stored pairing credentials permit subsequent sessions. Automatic first pairing is intended only for the trusted home-network policy. The host needs at least one Private Windows network, and each Pair/Start must arrive on an adapter Windows classes as Private; unrelated Public adapters (VPN, WSL/Hyper-V, a second NIC) no longer block sessions (RC2). The software does not silently change the Windows network category or disable a firewall. Multiple adapters can be inspected and the intended Ethernet/Wi-Fi route selected.

5.2 Starting

1. The user selects stream quality and presses Start on the client desktop page or Game Mode interface.
2. The client records ownership of temporary runtime settings, starts local SteamVR if needed and probes a physically ready headset with progressing display timing. From RC3, when a headset was verified in an earlier session, the client obtains the host lease (step 4) in parallel using that cached profile; if the fresh headset differs, the host session is stopped and restarted with the detected profile before streaming.
3. Fresh HMD identity, application render recommendation, effective refresh and stream preferences are validated. Auto settings resolve for this session without overwriting saved explicit preferences.
4. The companion obtains a host session lease. From RC4 it first wakes a sleeping host with Wake-on-LAN when the host's adapter address is known, and, with automatic bitrate, runs the network test once per host and adapter. The host applies the negotiated profile and registers its owned driver; any required owned SteamVR restart is completed before treating startup as ready. From RC4.1 the host companion writes the client's display (render size and refresh rate) for the driver before SteamVR starts, so the virtual headset already has the negotiated configuration and no restart is needed (patch 0015); the driver saves session.json and then records the resulting hash with the companion's nonce, so the companion can tell this startup update from ALVR's restart request (any other hash after the record, including the earlier one). The client's ALVR name lets the host add the lease's client address to ALVR's reconnection list. A host kept open for quick resume is reused when the new settings match.
5. The native client establishes the compatible ALVR connection, receives video configuration and enters streaming only when usable frames arrive. Connection, decoder and presentation states remain distinct in diagnostics.

On a Windows headset PC (1.0.0.7) step 2 finds SteamVR through OpenVR's registry and starts it with its own `vrstartup.exe`; a PC where SteamVR has never run gets a plain message instead of a timeout. The session-only Windows settings (Wi-Fi, power mode, keep-awake) are applied beside the start, as Bazzite's are (section 4.4).

Failures are surfaced by stage. A running process, visible headset icon or discovered host alone is not proof that video is flowing. Cancellation must release the host lease and stop resources started for the attempt. Restart timeouts should preserve a report rather than pretending that SteamVR stopped successfully.

5.3 Running and stopping

The companion protocol uses a 3-second heartbeat and 30-second host lease. A lost companion therefore has a bounded ownership window. From RC4.1 heartbeats to a `quality-4` host carry the client's stream state (fresh video or seconds without it since the stream was last up). While that report is current, the host restarts its owned SteamVR session when the stream stays down: 8 s with ALVR stuck in Disconnecting, 20 s while ALVR still shows the client Connected or Streaming, 45 s otherwise; at most twice per session, three minutes apart. ALVR itself retries the client's known address every second. Start requests are idempotent and an unrelated client cannot stop another owner's session. The protocol defines a 5-second stop grace; actual runtime shutdown is also subject to process/lifecycle checks.

Client Stop ends native streaming and closes the SteamVR runtimes managed for the session on both PCs. The host returns to waiting. With quick resume (RC4, client option, off by default) host SteamVR stays open for the chosen 5-60 minutes and closes afterwards, or at once on a host-page Stop; games on the host keep running meanwhile. The host PC is kept awake during sessions and may sleep a chosen time after the session ends (host option, off by default). Local runtime settings are restored from ownership receipts after stopping; unrelated user edits must not be overwritten. Closing a browser tab leaves the companion running. A runtime crash, OS shutdown or forced process termination may interrupt graceful cleanup and client-report transfer. On Windows (1.0.0.7), which has no SIGTERM, the

companion stops the native client by writing `quit` to its command pipe and waiting up to 8 seconds, then closes the pipe; SteamVR is asked to quit with `vrmonitor://quit` before only this session's verified processes are ended. The Windows power mode, keep-awake request and Wi-Fi settings come back at Stop, and the power mode also at the next start after a crash, from its receipt.

The host owns only the SteamVR it started: the vrserver processes it observed, and the runtime family's processes (`vrserver`, `vrcompositor`, `vrmonitor`, `vrstartup`) from that SteamVR installation created after its launch, by PID, image path and creation time. Stop asks the driver to shut SteamVR down gracefully and, after a timeout, ends only those identities. From 1.0.0.3 a new vrserver joins the session when it is that installation's `vrserver.exe`, created after this launch, while a process of this session's SteamVR still runs or while the monitor waits out a runtime loss. In a 1.0.0.2 report SteamVR started a second server 3 s after a 61.9 s start; the session ended and Stop refused to close SteamVR. Any other new vrserver still ends the session and is never stopped (SteamVR ownership changed). The host log says Host runtime interrupted only for a runtime that was ready or whose server exited, and says once when SteamVR has been starting for 30 s without the driver.

The RC4.3 test ended when SteamVR's compositor on the client crashed mid-session (vrserver: `vrcompositor disconnected`, nothing in the compositor's own log): nothing could be shown in the headset, VROSTBridge's reconnecting card included, and nine seconds later the native client was killed by a broken pipe (exit −13). From RC4.4 the native client and runtime probe ignore SIGPIPE; the client logs `STATUS SteamVR compositor frames stopped advancing (2 s)` and `... advancing again`, and stops by itself when the companion's command pipe (its stdin, when that is a pipe) closes, so it never runs on without the companion. During a session the companion records this PC's SteamVR server and compositor processes once fresh video arrives and then checks those exact processes (pid, executable, owner, start time) every second, so an unreadable SteamVR registry never looks like a crash: the compositor gone while the server runs, SteamVR gone as a whole, or a new compositor while the native client reports stalled frames counts as lost after 2 checks in a row; a compositor SteamVR started in place of a lost one is on probation for 4 seconds (longer than the client's 2-second stalled-frames line) and becomes the one watched only if this session's frames keep going. Nothing is judged while a Stop runs or has been asked for, after SteamVR asked the client to quit (`STATUS Local SteamVR requested this client to quit`), or before the processes were seen. A native exit with an error while SteamVR is lost takes the same path, and a `FATAL` line waits up to 15 seconds for SteamVR here to show it failed (a crashed compositor can stay in `/proc` while its core is written), with the error shown meanwhile; a clean exit just stops. It then says so with the time, closes the session on both PCs and starts it again (retrying while the host finishes stopping, for up to 60 seconds), and the pages and Game Mode show restarting and then restarted for two minutes. A Stop or Start by hand wins: the restart's own Stop and Start carry the count of actions by hand when it began and are refused, under the action lock, once that count has changed, a Stop asked for before the restart began (`session_cancelled`, checked under the same lock) prevents it, and a Stop by hand clears the restart message. A second crash within ten minutes stops the session with the explanation instead. The companion also logs the compositor's `coredumpctl` entry and the kernel's graphics lines (`journalctl -k`, last 5 minutes, filtered to GPU/DRM and `vrcompositor` lines, at most 40 lines) for the report, when this user may read them. On Windows (1.0.0.7) the same recovery applies, and the report takes Windows' own records instead: Application Error and Application Hang events for `vrcompositor.exe`, `vrserver.exe` and `VROSTBridgeClient.exe` from the last 10 minutes and display-driver resets (Display event 4101) from the last 5, read as XML so any Windows language works. VROSTBridge cannot prevent the compositor crash itself.

Do not launch another ALVR instance alongside VROSTBridge. The software avoids unowned driver/profile changes and may require the user to close a conflicting manually started runtime before retrying.

6. Optical geometry and display negotiation

6.1 Geometry contract

Use the physical runtime's per-eye raw projection tangents and complete eye-to-head transforms. Do not derive an HMD's projection from a nominal panel resolution, a generic symmetric FOV, an Index template or an IPD value alone. Stream resolution sets the render/encode budget, not the optical model.

The RC1.3 client maps OpenVR raw values to ALVR angles as follows: left = atan(rawLeft), right = atan(rawRight), up = atan(rawBottom), down = atan(rawTop). OpenVR's vertical naming convention is the reason for the bottom/up and top/down mapping. The pinned host reconstructs raw top from tan(down) and raw bottom from tan(up). The previous sign-flipped conversion mirrored an asymmetric vertical optical-centre offset; symmetric tests alone would not expose it.

Full eye position and orientation are preserved. Dynamic geometry updates, including runtime IPD events, must not associate a frame captured with old optics with unrelated new eye geometry. A bounded geometry history supports source-pose recovery. Invalid/nonfinite updates are rejected. Changing headset model or display mode should be done with the session stopped; IPD changes themselves are supported while running.

Reports retain static `view_geometry` on valid changes: raw tangents, transmitted angles, eye transforms and geometry revision. They do not continuously record live world tracking or gaze samples. The client submits side-by-side stream eye bounds through the local runtime, which remains responsible for physical lens distortion and display scan-out. RC1.3 does not swap eye images or invent Beyond lens constants.

6.2 Refresh handling

Ordinary readiness requires valid runtime geometry and advancing physical timing. A narrow Beyond exception accepts a reported 90 Hz / measured 75 Hz mismatch only after the evidence window and stability guards pass. It observes at least 1.5 seconds and 110 progressing frames with approximately 3% tolerance; invalid timing, counter changes and geometry/mode changes reset the observation. Index startup transition guards remain strict.

The effective refresh propagates into headset metadata, host negotiation and client frame budgets. The software does not force a SteamVR display mode or edit the user's refresh preference to conceal the mismatch. Physical mode selection, brightness, firmware and vendor-specific display controls stay with the local runtime/vendor tools.

7. Video quality and frame delivery

7.1 User controls

New preferences default to Auto codec, Automatic headset resolution, Auto foveated encoding, 10-bit colour and automatic bitrate (80 Mbit/s until a network test has run). The bitrate control accepts 10-300 Mbit/s. Codec, bitrate, stream size and the RC4 stream options are remembered per headset family. Existing saved settings are preserved, so an upgraded installation may show a different value. Increasing bitrate may improve compression detail but also consumes encoder, network and decoder capacity; it is not inherently a latency improvement.

Auto selects H.264 for Index and HEVC for Beyond; AV1 is an explicit choice that needs AV1 encoding on the host GPU and decoding on the client. Automatic headset resolution selects the 2016-pixel-per-eye-width budget for both headsets (High on an Index; Standard on a Beyond, since RC4.3, so a handheld headset PC keeps up). UI labels include meaningful per-eye dimensions once the headset is detected. Legacy setting identifiers remain stable to preserve existing preferences.

Choice / scope	Requested per-eye width	Constraint
Index Standard / legacy Custom	1440 with H.264; 2144 with HEVC	HEVC compatibility budget is labelled Custom
Index High / Beyond Standard	2016	H.264 or HEVC
Beyond Native (until 1.0.0.10 High)	2560, the panels' own	HEVC or AV1
Beyond Ultra	3200	HEVC or AV1
SteamVR recommendation	Local runtime application recommendation	Capped/aligned for selected codec

Widths and computed heights are aligned to 32 pixels. Height follows the runtime recommendation's aspect ratio. The H.264 width cap is 2048 per eye and HEVC cap is 4096 per eye; these are software bounds, not guarantees that every GPU can encode/decode them at the chosen refresh. The encoded frame is side-by-side stereo. The actual negotiated/decoded dimensions are authoritative and must be displayed separately from requested dimensions and physical panel pixels.

7.2 Pipeline policy

The baseline uses H.264, HEVC or AV1, SDR and unit gamma; HEVC and AV1 use 10-bit encoding when the client option is on (P010 surfaces, sampled as 16-bit on the GPU). Chroma is sampled at its signalled position and the 8-bit output is dithered. The client decodes with the matrix and range the stream signals. From 1.0.0.5 (patch 0018) the AMD path makes those signals exact: up to 1.0.0.4 it handed the encoder 10-bit RGB and left the conversion to it, which gave studio-range samples (black at 64 of 1023) in a stream signalled full range, so black showed as dark grey on OLED panels with 10-bit colour on. The host now converts to Y′CbCr itself (BT.709, full range, `ColorSetup` in `VideoEncoderAMF`), and at the start of every stream converts a test picture of black, white, red and mid-grey stripes with the same converter, reads the samples back and classifies their range and matrix (`VROSTBridgeColor.h`, tested on Linux by `host_color_tests`). The stream's signals (H.264 `FULL_RANGE_COLOR`, HEVC `NOMINAL_RANGE`, the input and output colour profiles, AV1's) follow what was measured; when the check fails or the encoder refuses the 10-bit settings the host streams 8-bit. Its log says `VROST_COLOR path=… bits=… range=… matrix=… verified=…` with the measured codes, and `VROST_COLOR_FALLBACK` why it chose 8-bit. The client logs the colour its decoder reports whenever it changes and shows bit depth and range in the video details, with a note when 10-bit was asked for but the host streams 8-bit. HDR is outside this client contract. FFmpeg hardware decode uses VAAPI on the intended Linux path; Vulkan handles import/conversion/presentation. From 1.0.0.7 the Windows client decodes with D3D11VA on SteamVR's GPU and copies each picture through a staging buffer into the same plane images, so conversion and presentation are the same (section 4.4).

From RC4.3 hardware-decoded frames take the fused frame pass (until 1.0.0.10 where the device allowed its 208 bytes of push constants; from 1.0.0.11 on every device, section 7.3): when a frame is decoded, its two planes are copied into images the client owns (`VulkanVideo::copyPlanes`, about 3 bytes per decoded pixel, then the VA-API surface is released as before); when it is shown, one compute pass (`fused.comp`) writes the image SteamVR reads, per output pixel: the re-aim homography (identity when not re-aiming), ALVR's foveation inverse with its slope (from RC4.4 the formula's analytic derivative, `foveationDecompressSlope`, instead of two more inverse evaluations), luma by Catmull-Rom from the decoded plane (from RC4.4 five bilinear taps, the four corner taps of RC4.3's nine dropped and the rest renormalised), optional contrast-adaptive sharpening of luma (4 more taps, the AMD FidelityFX CAS amount rule, at full strength where an output pixel spans at least one encoded texel, faded to nothing at half a texel and skipped below, so the reduced edges cost no sharpening taps), chroma bilinear at its

siting, colour conversion, dither and the 2-pixel fade outside the rendered view. RC4.2's path (convert.comp into a full-size RGBA image when decoded, reaim.comp when shown) remains the fallback, also chosen with the troubleshooting setting One-pass frame path off (`one_pass`, native `--two-pass`; no sharpening), and CPU decoding keeps its own path (on Windows, from 1.0.0.7, CPU-decoded planes take the fused pass too). From 1.0.0.7 to 1.0.0.10 a device that offered fewer than 208 bytes of push constants, down to the 128 every Vulkan driver must (some AMD drivers on Windows offer just that), took the same pass compiled as `fused-uniform.spv`, which read the eye re-aim rows from a small uniform buffer; from 1.0.0.11 every constant comes from a uniform buffer and one shader serves every device. Per frame the fused pass writes one full-size image instead of two and resamples the picture once instead of twice; `tests/fused_shader_tests.cpp` checks it against its CPU model and RC4.2's chain on lavapipe (under a 0.37° re-aim a detailed scene differs from the original by 0.64 levels in RC4.4, 0.72 with RC4.3's nine taps, against 2.21 for RC4.2's two passes; the analytic slope matches finite differences within 0.001). Sharpening (Off, Low, Medium, High = 0, 35, 70, 120 % unsharp amount; default Medium) applies at once (`--sharpening N`, stdin `sharpen N`); it is separate from foveated encoding, which decides how much of the picture is sent, and the setting's text says so. `TIMING` reports `frame_pass`, `fused_frames`, `fused_skipped` (refreshes with no free output image, when SteamVR shows the previous one) and `sharpening`, and from 1.0.0.11 `frame_pass_gpu`, `plane_copy_gpu`, `shown_scale`, `shown_eye_width` and `visible_share`. Explicit CPU compatibility decoding is available for diagnosis, not the performance target, and turns foveated encoding off because it cannot undo it.

Packet, decode and presentation queues are bounded. Decode and conversion work are separated so slow GPU work does not unnecessarily block receive/decode. Selection favours timely usable frames with their correct source poses. Frame replacement must respect GPU ownership and completion; resource reuse cannot race a compositor still using the image.

Brief interruptions can reuse a valid remote frame: the implementation distinguishes fresh frames below about 100 ms from a held-frame window below one second, in which the last frame is shown re-aimed to the current head rotation (`core/stream.hpp`; 250 ms until 1.0.0.3, which turned every stall a little longer than that, such as a lost frame waiting for its keyframe on a busy headset PC, into grey: about 11 seconds in one Beat Saber session of the 1.0.0.3 test). The Waiting for video card still shows after half a second. Longer loss or invalid tracking uses the neutral scene. This avoids intentionally dropping into the local Home environment. Runtime/GPU failures can still expose SteamVR's own safety or error visuals; the bridge cannot promise to suppress those.

H.264 streams use CABAC entropy coding from RC3. A client encoder priority selects the host encoder preset (lowest latency by default, or balanced quality). The game-audio jitter buffer is 30 ms for a wired client and 50 ms otherwise. The software does not silently lower stream resolution or visual quality mid-session. From 1.0.0.11 the headset PC may show each eye smaller (90 %, then 80 %) while its graphics hold the stream below refresh, with Keep the frame rate when this PC falls behind on; the page, the log and the timeline say so, and the stream itself is unchanged (section 7.3). Performance work should first address avoidable queueing, missed deadlines, clock/pose association and recovery. Compare quality presets explicitly and retain both the selected and actual negotiated configuration in reports.

7.3 Measurement and current issue

Frame age is an internal pipeline metric (time since the pose was sampled), not a measured motion-to-photon latency. From RC2 each one-second window is also classified as `full`, `client_throttled` (local SteamVR running the app below refresh, including its own throttle count), `source_limited` (fewer frames arriving) or `late_or_paired`, and class changes are logged as `PRESENTATION_CADENCE`. Evaluate its distribution together with unique presented frames, display refresh, repeated/late frames, decoder/conversion times, network loss and host encoder/render timing. Once-per-second summaries can hide short stalls and must not be presented as per-frame percentile measurements.

From RC3 the host keeps a 5-second window of ALVR's per-frame breakdown (render, encode, network, decode, display wait) and the client shows it as a session quality line, including the circular mean and

spread of how long decoded frames wait for the headset's frame pickup.

Foveated encoding (RC4; patches 0010 and 0012)

Presets set ALVR's per-axis centre size and edge ratio: Subtle 0.70 x 0.65 at 2:1, Balanced 0.55 x 0.50 at 3:1, Eye-tracked 0.40 x 0.40 at 4:1 (from RC4.3 the eye-tracked region is Small 0.30, Medium 0.40 or Large 0.55 and the outside More detail 3:1, Balanced 4:1 or Lightest 5:1, within what RC4.2 hosts accept; at 2560 per eye Medium/Balanced encodes 1408 per eye, 70 % fewer pixels). Auto chooses Subtle for Index and Beyond and Eye-tracked for a Beyond with calibrated gaze forwarding; unknown headsets stay off. The host compresses before encoding; the encoded eye size follows ALVR's negotiation (32-pixel aligned) and the centres for every frame travel with it. Without fresh gaze the RC4 driver centres the region on each eye's straight-ahead direction computed from that eye's field of view and pose (optical centre); with fresh gaze (under 150 ms old) it centres the region on the gaze direction, snapped to the negotiation grid and held inside a 2.5 % dead band. From 1.0.0.5 (patch 0019) it moves only when the gaze leaves a comfort zone of `gaze_comfort` × half the region's size around the region's middle (default 0.5, accepted 0 to 0.8; never less than the dead band) and then goes straight to the gaze: tracker noise and small eye movements crossed the dead band every few frames, so the region's edge kept shifting where the user could see it. In a simulated minute of gaze at 75 frames a second (fixations of 150-450 ms, jumps of 2-15 degrees, 0.5 degrees of noise) the 40 % region moved 0.85 times a second instead of 3.17, with the gaze always at least 9 % of the picture inside its edge. The companion writes `gaze_comfort` only for a driver whose build manifest lists patch 0019 (feature `gaze-comfort`). From 1.0.0.4 (patch 0017) a pause in the gaze (no new sample for 150 ms: a blink, or the eye tracker skipping pictures) keeps the region where it was for `gaze_hold_ms` more (stream options; default 1000 ms, at most 5000) before it returns to the optical centre. Up to 1.0.0.3 it returned at once: in the 1.0.0.3 test the region was in the optical centre for 39 % of the frames of both sessions, one pause lasted 20 seconds, and the user could look straight at the reduced area. The driver's `VROST_GAZE_FOVEATION` line counts `gaze_frames`, `held_frames`, `fallback_frames`, `pauses`, `longest_pause_ms` and `returns`; the companion writes `gaze_hold_ms` only for a driver whose build manifest lists patch 0017 (feature `gaze-hold`), and the pages count held frames as following the eyes. The client's conversion shader implements ALVR's inverse (`stream.wgsl`) per eye. A CPU reference and a software-Vulkan test compare the shader with the reference and with the original image through the host compression formula.

1.0.0.3, the reduced area. ALVR's compression pass took one bilinear sample per encoded pixel, from up to five source pixels per axis, so detail aliased and sparkled as the head and the gaze moved. Patch 0016 gives the composition texture (or, with ALVR's colour correction on, that pass's output) a full mip chain, regenerated each frame (`GenerateMips`), and binds an anisotropic (16×), clamped sampler for the compression pass; the sampler bound before is restored afterwards. If the mip chain cannot be created the driver samples as before, and its log line `VROST_FOVEATION_PREFILTER composition_mips=0|1` says which. On the client, the fused pass resamples luma with Catmull-Rom (five bilinear taps) where an output pixel spans at least one encoded texel, and with an exact cubic B-spline (four bilinear taps) where it spans half a texel or less. The two blend linearly in between, using the same factor that fades out sharpening. Catmull-Rom's overshoot drew light and dark fringes in the enlarged edges; the B-spline has none. The tests measure a third of the change for detail moving under the pixels and no overshoot at a step (`tests/fused_shader_tests.cpp`, test 6).

From RC4.3, while a foveated stream plays, the native client prints `FOVEA` lines ten times a second (thirty from 1.0.0.3): the shown frame's region size and per-eye centre (`0.5 + (1 − size)/2 · shift`, mirrored for the right eye), the edge ratio, the encoded and view sizes and, with a gaze sample under 100 ms old, the gaze point in each eye (the host's `eye_image_position` projection). The companion keeps only the latest in memory (`launcher/fovea.py`): it never enters the log, journal or reports. The pages poll it through the local `/fovea` endpoint eight times a second (1.0.0.3: twenty times a second, drawn at the screen's rate with the gaze point and region gliding to each new position, a one-second gaze trail, and the reduced area and sharp region as coarse and fine squares; Game Mode redraws twenty times a second) for the Eye-tracked foveation panel (two eye pictures with the sharp region and the gaze point, plus

region size, outside resolution, pixels sent and saved, share of gaze-centred frames, gaze sample rate, frame pass and sharpening), shown only while eye-tracked foveation runs; Game Mode has the same view (Stream & headset → Eye-tracking view). The client shares the view with the host page (1.0.0.3: ten times a second, five before) only during its own session, only with a host that lists `fovea-view`, through the authenticated session-link `/fovea` request (lease owner only, at most about 12 per second, validated, memory only). Host reports keep the driver's `VROST_GAZE_FOVEATION` counts but no longer its centres. From RC4.4 the eye-tracked Sharp region and Outside the sharp region settings show whenever Eye-tracked is chosen (RC4.3 hid them until the region followed the eyes); when the region does not follow the eyes a note on the page and a row in Game Mode say why, from the companion's `eye_state`. In RC4.5 (section 9) the states are:

- `off-here`: Eye tracking off; the fixed Balanced region.
- `no-cameras`: a headset without eye cameras.
- `setup` / `calibrate`: Baballonia not set up, or not calibrated; Balanced.
- `restart`: not ready when this session started.
- `starting`, `cameras` (permission, not found or no picture), `closed`, `eye-error`.
- `host-off`: the host's driver keeps the region centred; only a pre-RC4 driver does. RC4.4's advice to turn eye tracking on at the host was wrong: foveation never needed the host's switch.
- `no-gaze`: the host follows gaze but none arrives.
- `cpu`: CPU decoding, so no foveation.

`centred` modes count only once the host has reported `gaze_foveation`.

Network test, automatic bitrate and pacing (RC4; patch 0009)

The network test sends frame-sized UDP bursts from the host to the requesting client only, at rising rates, between sessions. The client measures loss, burst spread and jitter and recommends 70 % (Ethernet) or 60 % (Wi-Fi) of the highest clean rate, within the codec limit; the result is kept per host and adapter. The host paces each video frame at the client's link rate (92 % of an Ethernet adapter's speed; for Wi-Fi, from RC4.1, the clean test rate on that path only, because real Wi-Fi throughput is often under half the radio's PHY rate; RC4 used the larger of the two), unpaced without a Wi-Fi test result, when that would fall below 1.25 x the bitrate (RC4: 2 x) or for a 10 Gbit/s client. From RC4.1 the test ladder is 80, 150, 200, 250, 300, 400, 500, 600, 750 and 900 Mbit/s. From 1.0.0.4 a wired link's pace is also capped at the highest test rate below the first step with loss (`lossless_mbps`, kept with the test result; none when no step lost anything, so a faster link is not held to the test's highest step): in the 1.0.0.3 test a USB Ethernet adapter linked at 1 Gbit/s carried 500 Mbit/s without loss but lost 12 % at 750, while each frame was paced at 920 Mbit/s and about one frame in every 3 seconds was lost, each costing a keyframe and three or four frames. A wired path whose saved test predates the field is tested once more before its next session; without any test a USB adapter (its network device under a USB bus) is paced at 60 % of its link speed. The test itself still sends at the full link pace, so every step is measured. From 1.0.0.5 the test measures the network rather than the companion: in the 1.0.0.4 test the same adapter, which had carried 500 Mbit/s without loss, lost three packets at 80 Mbit/s and its frames looked late from 150 Mbit/s, so no step was clean and every later session ran at 30 Mbit/s for the result's 30 days. The receiver is a thread of the companion, so datagrams waited for its turn: frames looked spread out, and datagrams that had arrived were dropped from a full socket buffer and counted as lost. On Linux the receiver now takes arrival times from the kernel (`SO_TIMESTAMPNS`) and counts the socket's own drops (`SO_RXQ_OVFL`, `local_drops`; `network_loss_pct` leaves them out, and a step whose frames were lost only here is still judged on the frames that arrived, if at least half did); while the test runs other threads switch every 0.5 ms instead of 5. Windows offers neither socket option, so the Windows client (1.0.0.7) stamps arrivals with the companion's own clock and cannot count its buffer's drops. A warm-up step at the lowest rate comes first and is not judged. Drops reported with a step's first datagram belong to the step before it (its last frames and end markers, dropped while the receiver stalled), and frames sent

after the last one seen in a step without an end marker count as lost from the host's own count (`frames_sent`, `unseen_tail_frames`). The pace limit (`lossless_mbps`) is the step below the first that lost 1 % of its packets or more on the way (`CAPACITY_LOSS_PCT`) instead of the first with any loss. On a wired link the automatic bitrate is at most 40 % of the pace (`SEND_FRACTION`, limit `latency`), so an average frame arrives within 40 % of a frame interval. A test that finds no clean rate, or receives nothing, counts as a failure: until three in a row an earlier clean result on the path stays in use, and the path is tested again before its next session (`needs_retest`). Optional gradual loss recovery enables encoder intra refresh (NVENC session settings; AMF via patch 0011); the client relaxes its keyframe requests (patch 0008: a keyframe after three consecutive lost frames) only after the host confirms intra refresh is active. From 1.0.0.6 lateness is what the way to the client added to each frame: every datagram carries its send time, the receiver keeps each frame's first and last, and a step is late when the 95th percentile of (arrival spread − send spread) reaches 0.6 of a frame interval (`network_spread_p95_ms`, `send_spread_p95_ms`). In the 1.0.0.5 test the host's sender, a Python loop at about 50 µs a datagram, took 9.5 ms to write a 150 Mbit/s frame: the step was judged late though nothing was lost up to 300 Mbit/s, and the bitrate was 50 Mbit/s. A saved result limited by lateness without these fields is tested again before the next session.

Steadier streams (1.0.0.8; client and host companions)

The 1.0.0.7 test's reports showed the cost of the 1.0.0.6 network test's higher results. Against the evening before, at the same 2432 × 1216 picture: automatic bitrate about 195 Mbit/s (about 50), paced at 500 Mbit/s (300); the headset PC's hardware decode 7.8 ms a frame (4.6); network delay 11-12 ms (7); the frame-timing lock steering in 5 of 48 updates (34 of 34); a lost frame every 12 s (27); the host predicting 63 ms ahead (44). The same session's GPU stayed at 800 of 2700 MHz, 54 % busy at a flat 20 W, the base stations were found 19.6 s after SteamVR started, and a fresh calibration put the gaze 15-20° right of straight ahead. Decisions:

- **A picture ceiling** (`launcher/stream_budget.py` `quality_ceiling_mbps`): automatic bitrate is at most 0.22 (HEVC), 0.32 (H.264) or 0.18 (AV1) bit per encoded pixel per frame, within 20-300 Mbit/s. A link's capacity is a limit, not a target: beyond a few tenths of a bit per pixel the picture barely improves while every frame costs more to send and decode.
- **Pacing headroom** (`launcher/link_test.py` `PACING_HEADROOM`): wired pacing at 70 % of the test's loss-free rate instead of all of it, so other traffic or a busy moment cannot tip frames into loss. The test's own pace is unchanged.
- **A governor** (`BitrateGovernor`), per streaming second from the native `TIMING` line after 8 s: down 20 % on 2 losses or 5 troubled seconds in 10 (troubled: a loss; decode mean over 42 % or 95th percentile over 65 % of the frame interval; or late frames with the packet-interval 95th percentile over 1.45 intervals), never below 35 % of the ceiling or 20 Mbit/s; up 10 % after 40 clean seconds, that wait doubling with each step down to 160 s. A step taken for decoding alone is undone when decoding is not at least 7 % faster 6-14 s later, and slow decoding then stops counting for the session, so a decoder whose time follows the picture size does not cost picture for nothing. The bitrate is the one lever the client has during a session; size and codec stay fixed until the next Start.
- **Live change on the host** (feature `bitrate-control`): heartbeats carry the wanted bitrate; a lease starts at the session's own bitrate; the host applies a change at most every 2 s, while the lease's stream runs, through ALVR's `session/values` interface, which ALVR's bitrate manager reads for every frame (no patch, no restart). A failed change is asked for again up to three times. The governor runs only against a capable host, so an older host keeps the starting bitrate and the page says so.
- **Performance balance, Bazzite** (`launcher/session_tuning.py`): only while video streams and only through steamos-manager's session-bus interface. The TDP goes to its maximum on the charger. The manual GPU clock is raised (45 % of the maximum, then 200 MHz at a time, at most 75 %) when frames run late with the GPU at no more than half its maximum and at least 35 % busy for two samples, and lowered when the processor is held below 75 % of its minimum or eye tracking falls below 70 % of its settled rate. Everything is restored at Stop, or from a receipt after a crash, and a value changed elsewhere is left alone. It is the automatic form of the RC4.4/1.0.0.4 advice, between

1.0.0.7's GPU held at 800 MHz and 1.0.0.3's processor starved by a GPU fixed at its maximum.

- **Base stations** (`launcher/base_stations.py`, `launcher/BaseStations.ps1`): the SteamVR 2.0 power command over Bluetooth LE from the headset PC (BlueZ through `busctl` and `bluetoothctl`; Windows' Bluetooth API from PowerShell 5.1), on at each Start and off as the client closes (a separate process, so closing is not delayed), with buttons. Only `LHB-` devices are contacted, nothing is paired, and the last result is kept for the card.
- **Gaze centring** (`gaze_provider.GazeCentre`): the median of the session's raw gaze, beyond a resting band of ±3° yaw and −12° to +4° pitch, approached 30 % per update and capped at 25° and 15°, taken off every sample and remembered per calibration (two angles per model file name, latest 8). **Use previous calibration** (`BaballoniaManager.calibrations`, `use_calibration`) points `EyeHome_EyeModel` at the model before the current one, only while Baballonia is closed.
- **Use recommended settings**: the settings above in one action at the next Start, with gradual loss recovery on Wi-Fi only.
- **Defaults:** the three new switches are on, because each acts only when it applies (a streaming session; Bluetooth with 2.0 base stations; a calibration whose middle sits off-centre). Each applies at once, also during a session.

Lost packets rebuilt (1.0.0.9; patch 0021)

In the 1.0.0.8 test motion and picture were much steadier, and the stutters left were mostly lost frames: a frame whose datagrams lost one piece was lost whole, and the keyframe that followed froze the picture for several frames. Each loss also counted against the bitrate governor, which stepped down twice. Decisions:

- **Parity, not retransmission.** The host adds one XOR parity piece per group of up to 10 data pieces of a frame, the groups interleaved across the frame so a short run of losses spreads over several groups. The client rebuilds one missing piece per group as soon as the rest of the group arrives; nothing waits for a round trip. Game audio, a few small pieces per packet, is sent twice (group size 1).
- **Backwards compatible both ways.** Parity travels on the data stream's ID with the top bit set. An older client has no queue for it and drops it like any unknown stream, and an older host sends none. The companion turns parity on (`fec_video_group` 10, `fec_audio_group` 1 in `vrostbridge-stream.json`) only for a driver whose build manifest lists the patch.
- **Cost:** about a tenth more video data, paced with the frame, and the audio twice (about 2.3 Mbit/s for the second copy, since each parity datagram carries a full piece). It is counted on the client (`net_fec_video` on `TIMING`, Rebuilt on the way on the page, `repaired` in the timeline).

Uneven arrival checked (1.0.0.10; client only)

In the 1.0.0.9 test no frame was lost in more than twenty minutes on Wi-Fi, yet the governor stepped from 40 down to 20 Mbit/s because frames arrived unevenly. The cause was the link, not the bitrate: short pauses of the Wi-Fi radio (most about 100 ms, the beacon interval Wi-Fi power saving keeps) bunched frames up at any bitrate, and at 20 Mbit/s they arrived no more evenly than at 40. Decisions:

- **A step for uneven arrival alone is checked** (`BitrateGovernor`), as one for decoding alone already was. The packet-interval 95th percentile of each streaming second at the current bitrate is kept (up to 60 s); a step taken with at least 20 s of it, and with no losses or slow decoding in the window, is followed by 12 s of the same measure after the host has applied it. When their median is not at least 7 % below the median before the step, the bitrate goes back up and uneven arrival stops counting for the session. The comparison is with the bitrate's usual seconds, not the troubled ones that triggered the step, which would look better afterwards in any case.
- **Missing frames are not uneven arrival:** a second the client classes as source-limited (fewer than 70 % of the refresh rate's frames arrived, from the host or the network, as while a game loads) no longer counts as uneven arrival, nor in the comparison.

- **The pauses are left to the user's network settings:** lost frames and slow decoding still step the bitrate down, and the page's Turn off Wi-Fi power saving (between sessions, since NetworkManager applies it by reconnecting) removes the usual cause on Bazzite.

A lighter picture on the headset PC (1.0.0.11; client only)

In the 1.0.0.8 test a Beyond streamed 2560 per eye at 75 fps with the headset PC's GPU about half busy. SteamVR measured about 7 ms of GPU work per 13.3 ms frame, some 4.4 ms of it the client's frame pass and plane copy, the rest its compositor. With eye-tracked foveation a 2560 stream is about as big to send and decode as Standard's with fixed foveation (1408 × 1408 per eye after foveation, against 1408 × 1344), so the frame pass, whose work grows with the shown size, was the part to make lighter. Decisions:

- **Per-frame foveation constants.** On the integrated GPUs typical of headset PCs, the driver's own compiler statistics (RADV, `VK_KHR_pipeline_executable_properties`) showed the pass bounded by arithmetic, mostly the foveation inverse's terms: about ten divisions per axis and pixel, from values that are the same for the whole frame. `fusedAxis` and `fusedFrameConstants` (`client/device/conversion.hpp`) work them out per frame, eye, axis and region of the axis in double precision, with the texture size, the right eye's mirrored axis and its offset folded in: a reduced edge becomes `r = sqrt(max(a m + b, 0))`, texel `= c r + d`, the sharp centre a straight line, and an edge ratio of 1 or an empty edge leaves the centre only. A pixel then needs one square root and one reciprocal per axis. Each eye is its own layer of workgroups, so everything that depends on the eye is the same across a workgroup. The compiler's estimate of the work per pixel fell by about a third, in the reduced periphery and the sharp centre alike. A gather-based neighbourhood for Catmull-Rom was left out: it trades image reads for arithmetic in a pass bounded by arithmetic.
- **One shader.** The constants (544 bytes, std140: colour, sizes, and per eye the re-aim rows, the foveation constants and the texel bounds that keep taps inside the eye) are written into a uniform buffer per output image while that image is free, so the pass uses no push constants and `fused.spv` runs on every device.
- **Only what the lenses show.** The client reads SteamVR's visible-area mesh (`GetHiddenAreaMesh`, inverse) and the outline of its lens distortion along the panel's edge (`ComputeDistortion`, three colours) once at start; `client/device/visible_area.hpp` keeps each output row's span per eye inside both, widened by 3 % of the eye's size, since a late frame is shown again turned to a newer head pose. The spans are in a storage buffer per output image and the pass returns outside them; the image is cleared to black once when it is made, when the spans change and after the re-aim pass has filled it. SteamVR documents the distortion's coordinates but not which way up the mesh is, so `alignMeshes` compares the two on a coarse grid: the mesh is used the way up that leaves less of the outline out, and left out when it leaves out more than 40 % of it either way; without an outline it is used both ways up, so nothing it shows is skipped whichever way it is meant. Only the one-pass frame path skips pixels. A description that would leave under half of either eye is ignored. Skip what the lenses never show (Preferences, `lens_area`, native `--full-frame`) turns it off.
- **Showing smaller instead of half rate.** `launcher/output_scale.py` reads the client's TIMING seconds: after 3 within 10 in which SteamVR showed the stream below refresh (`client_throttled`) while frames arrived and the headset was tracked, with SteamVR's GPU time per frame at least 75 % of the refresh interval, the companion asks the client (`scale N` on its standard input, applied from the next decoded frame) to show each eye at 90 %, and at 80 % if that happens again after 10 s to settle. After 60 s without such a second and with the median GPU share under 60 % (at least half of those seconds with a GPU time) it goes back up a step. A step up that falls behind within 30 s keeps the smaller size for the session; one that falls behind within 5 minutes doubles the wait before the next try, and three of those keep it smaller too, so a PC at its limit does not swing back and forth. Seconds behind with GPU time to spare, or without SteamVR's GPU time, do not count: a smaller picture might not help them. The companion chooses a size and sends it in one step under a lock the switch also takes, so turning the switch off (`scale 100`, sent at once) cannot be overtaken by a size chosen just before. The stream is unchanged, so the detail still arrives and is resampled once. Keep the frame rate when this PC falls

behind (This PC, on, applies at once); the page shows Shown at …, the log a `SCALE` line, the timeline the size.

- **Measured.** GPU time stamps around the pass and the decode-time plane copy (a pair per output image and per copy, each written once the work before it is complete, read back when it is next used) give `frame_pass_gpu_*` and `plane_copy_gpu_*` on `TIMING`; the page shows the pass's time beside the frame rate and the timeline keeps it with SteamVR's own GPU time per frame.
- **Native.** The Beyond's 2560 size is called Native, the panels' own resolution (the setting's identifier is unchanged). Without eye tracking it carries about 60 % more pixels than Standard, so Auto still starts a Beyond at Standard.

`tests/fused_shader_tests.cpp` compares the pass with its CPU model (results within a hundredth of a level of 1.0.0.10's) and the per-frame constants with ALVR's formula; `tests/visible_area_tests.cpp` checks the spans and the mesh's orientation check; `tests/video_pass_tests.cpp` runs the client's own `VulkanVideo` with a stand-in SteamVR (`tests/openvr_stand_in.hpp`, generated from the OpenVR headers): its buffers, spans and clears, a smaller shown size and the time stamps, from decoded planes on Linux (the VA-API import itself is not run) and, through the Windows client's own upload of CPU-decoded frames, under Wine. The output scale's rules are tested with made-up TIMING seconds. None of it has run on hardware yet.

Follow-ups from the first session of 1.0.0.11 (1.0.0.12)

The first session of 1.0.0.11 on hardware (Beat Saber at Native with eye-tracked foveation, wired, a handheld headset PC on Bazzite) held 75 new frames a second, with the sharp region following the eyes for 98.5 % of frames and five frames lost in nine minutes. The automatic bitrate went from 65 to 30 Mbit/s in the first 80 seconds, while the headset PC's GPU sat at 800 MHz and the game was starting; the companion held the GPU at 1300 MHz four and a half minutes in, but three steps down asked for 160 clean seconds before a step up, and the rendering PC's stalls (40-80 ms without a frame about every half minute, its capture timing out on SteamVR's frame) broke every run of them. Decisions:

- **A fresh start after a raised GPU clock** (`BitrateGovernor.fresh_start`): when the session tuning raises the clock, the steps down so far may have been the graphics', so the bitrate goes back to the session's starting bitrate, the next 8 seconds are not judged and the patience for a step up is the first step's (40 s) again. The patience now counts the steps down since the start or the last fresh start.
- **Frames that never arrived are not the bitrate's:** a second in which at least 1.5 frames never arrived (the refresh rate less `packet_fps`) with none lost on the network is neither late nor uneven for the governor and leaves its arrival gap out of the arrival check, since no lower bitrate brings frames that were never sent. The session tuning does not count such a second as late either; SteamVR's throttling and a frame the client could not show still count.
- **The clock the last session needed** (`launcher/session_tuning.py`): the clock a session holds is remembered per PC (`gpu-clock.json`, with the GPU's maximum clock and whether it was on the charger) and held when the next session's video is up, while the GPU's level is automatic and the GPU and power source match; a session that sets it back down, or in which it is changed elsewhere, forgets it.
- **Add-ons:** `host/addons.py` lists only drivers registered from outside SteamVR. Its list of SteamVR's own drivers had gone stale: it called indexcontroller, frame_hmd, frame_controller, prism, roy and tundra_labs third-party and suggested turning them off, Valve's Index controller driver among them, while the streamed controllers present themselves as Index controllers.

`tests/test_vrostbridge_1012.py` replays the 1.0.0.11 session's stall seconds through the governor, checks the fresh start, the remembered clock against a stand-in for steamos-manager and the add-on list against stand-in SteamVR folders. The rendering PC's stalls themselves are not addressed: why SteamVR's frames reached the capture late is not known yet.

Encoder output and late frames (RC4.1; patch 0013)

The AMF encoder path forwards every finished frame, oldest first, on each submission and waits (within one frame's budget per submission) only until a slot is free and the frame just submitted is out. Stages are never asked for output nobody waits for, the driver holds a 1 ms Windows timer period while encoding, and a frame in flight for 250 ms resets the encoder. Upstream ALVR forwarded at most one output per submission, so a single slow frame left the encoder one or two frames behind for the rest of the session. The client's late-frame catch-up waits at most min(3 ms, a quarter of the refresh interval, time to the compositor's deadline minus 2 ms) for a frame that is just late (RC2-RC4: 1.5 ms).

Even pose timing and frame re-aim (RC4.2; client only)

The host renders each frame with the newest head pose it has received and extrapolates it by one measured delay, so the poses behind consecutive frames must be evenly spaced in time. RC4.1 sampled poses every 4 ms on a free-running timer, which does not divide a 75 Hz frame: consecutive frames used poses 12.2 or 16.2 ms apart in a repeating pattern, and head-locked content shook while the head moved. From RC4.2 the client samples on a grid locked to the local display: N evenly spaced samples per refresh (spacing at most 3.5 ms, so four at 75 and 90 Hz), re-anchored to the measured vsync once a second. When the host's pick point sits on a sample (its frames alternate between neighbouring samples, more than 5 % of picks in about two seconds), the grid shifts by half a spacing, at most eight times per session.

Before submission the client can re-aim each frame: a compute shader maps every output texel through the rotation-only homography from the pose the compositor will display with (the pose `WaitGetPoses` returned) to the frame's source pose, per eye, with a 9-tap Catmull-Rom filter, and submits the result labelled with the display pose. Eye translation is not corrected (it needs depth). The pass records on the render thread from its own command pool into up to four output images, each reused only after the compositor has finished with it; when it cannot run, the frame is submitted unchanged. An alignment check labels every other second's frames with a pose turned 3° for 10 seconds: if the picture jumps, the local compositor already re-aims submitted frames and the client's pass is switched off. The `TIMING` log line reports re-aimed frames, fallbacks and angles, the grid and `pose_step`.

Headset-PC graphics limit (RC4.3, power limit RC4.4; client only)

The RC4.2 test streamed 2560 × 2560 per eye (HEVC 10-bit) to a Beyond from a headset PC with integrated Radeon graphics: frames arrived at 75 per second, but SteamVR on that PC kept running the stream at half rate (37.5 fps) for seconds at a time and latency rose from about 42 ms to 70-107 ms; 2016 per eye held 75 fps. From RC4.3 the companion counts the native `cadence` class per streaming second, ignoring the first 5 seconds of video and seconds when frames did not arrive at the refresh rate or the headset was not tracked with input focus; 8 `client_throttled` seconds in the last 30 mark the limit. It then pauses re-aim on the two-pass path (a full-resolution GPU pass per frame there; the saved setting is unchanged; with the fused pass re-aim costs nothing extra), shows a short line in the headset for 12 seconds on the session's first limit and a warning on both pages with the applicable advice: one-tap use of the headset's own smaller size (Standard on a Beyond, High on an Index; saved for the next Start, allowed during a session), the SteamVR reprojection repair when it is off, or closing other programs and checking for overheating. The limit clears only after a whole clean window. Auto resolution starts the Beyond at Standard (2016); Native (until 1.0.0.10 High) and Ultra are explicit choices. From 1.0.0.11 the picture is also shown smaller within seconds while the stream continues (section A lighter picture on the headset PC), which often answers the limit before this advice is due.

In the RC4.3 test (and both RC4.2 tests) the headset PC's integrated GPU (rated up to 2700 MHz) stayed at 800 MHz in every sample while it was about 58 % busy and the chip drew about 15 W, plugged in: SteamVR held the stream at 37.5 fps for 407 of 412 streaming seconds, so the host predicted head and controller poses about 96 ms ahead, felt as jitter. A power limit (TDP) or a fixed GPU clock, not the GPU, was the limit. From RC4.4 the companion samples the busiest GPU every 5 seconds during a session (`launcher/graphics_limit.py` `GpuClocks`: current clock from the sensors or the selected `pp_dpm_sclk` level, maximum from the highest level, load and power); with at least 3 of the last 6 samples, a median load of 40 % or more at a median clock at or below half the maximum marks it held back. The graphics warning

then puts that first with the measured numbers (Its GPU ran at 800 of its 2700 MHz while busy, at about 15 W: a power limit (TDP) or a fixed GPU clock is holding it back. With the charger connected, raise the TDP limit as far as it goes and leave the GPU clock on automatic (Steam's Game Mode: Quick Access menu > Performance; on the desktop, your power or TDP control), then start again.; up to 1.0.0.3 it named one handheld tool), leaves out the overheating advice, the headset line reads GPU held at 800 MHz · raise the TDP limit and the Game Mode session card says the same; the smaller size is still offered after it. During a session the latest sample is also shown live (`graphics.gpu`: the client page's fresh-video line, the Game Mode session card and the headset quality line), so a TDP change can be watched as it is made.

In the 1.0.0.4 test Ultra (3200 per eye on a Beyond, whose SteamVR on the headset PC recommends 2712) still fell behind. From 1.0.0.5 a stream larger than the runtime's recommended size is shown at that size (`shownEyeSize`, `shown_size_contracts`): the fused pass scales the decoded picture down in the pass that already converts it, so the headset PC draws no more than SteamVR asks for; the client log, the video details (shown at …) and the session's `VIDEO` line say so. When the graphics limit coincides with a GPU held back at a fixed clock that steamos-manager can set back, the warning offers Set the GPU clock to automatic (`gpu-auto`) as well.

The 1.0.0.3 test showed the opposite case. With the GPU clock fixed at its maximum (2700 MHz) under a 28 W limit, every processor core ran at about 540 MHz for a whole session, below the processor's own 1100 MHz minimum: Baballonia, which runs its model on the processor, fell from 54 to 12 eye-tracking samples a second, and the stream's own work stalled for 40-60 ms every few seconds. From 1.0.0.4 `launcher/power_balance.py` `CpuClocks` reads the same samples and marks the processor held back when its median clock is below 75 % of its own `scaling_min_freq` in two samples running (the system never asks for less than that minimum, so running below it means the firmware's power or thermal limit is holding it back, on any processor; without a reported minimum nothing is shown). The cause is `fixed-gpu-clock` when the GPU's performance level is `manual`, or its clock is at 90 % of its own maximum or more while it is less than 60 % busy, and `power-limit` otherwise. `advice()` words it with the measured numbers: for a fixed clock, set it back to automatic or fix it at 60-75 % of this GPU's own maximum; for the power limit, raise the TDP limit with the charger connected and leave the GPU clock automatic. `SteamOSPower` reads steamos-manager's `GpuPerformanceLevel1` and `TdpLimit1` properties on the session bus (`busctl --user`, at most every 30 s), where a PC has that service (SteamOS and the distributions that use it for Steam's Performance settings and their desktop TDP controls, such as Bazzite), and sets `GpuPerformanceLevel` to `auto` only when the user chooses Set the GPU clock to automatic (action `gpu-auto`, on the client page and in Game Mode under This PC). The warning shows on the session card, the client page and Game Mode, once as a short line in the headset, and in the log as `POWER_BALANCE`. Baballonia also runs one step lower in priority (`nice -n 5`), so the stream's own work goes first when the processor is short.

Frame-timing lock (patch 0007)

The pinned virtual vsync free-runs from the moment a client connects, so its phase against the physical panel is random per session and drifts with clock skew. Patch 0007 steers it from the client's per-frame `rendering` statistic (decoded frame to headset pickup), using circular statistics and a bounded PI controller, so frames land a margin (default 2.5 ms plus twice the measured spread) before pickup. It acquires in one step, then corrects at most 0.5 ms per update. The host companion enables it per session through `vrostbridge-timing.json` beside `session.json`, only when the installed driver's build manifest includes patch 0007; absent or false keeps upstream behaviour. Offline closed-loop simulation shows zero repeats or drops at every initial phase with ±300 ppm drift; the RC3 and RC4 host drivers include it; physical confirmation is pending.

The latest Beyond report includes periods near 75 unique frames/s and periods near 37 while the host remains near 75. Projection verification is the immediate gate; intermittent half-rate client presentation is a separate follow-up. Do not claim that raising bitrate or changing transport will resolve it without stage-level evidence. Earlier left-eye artifacts were reproduced locally by the user without VROSTBridge and should be investigated separately from bridge-only symptoms.

8. Tracking, input, menus and play area

Head and controller poses originate from the physical local runtime and travel to the host's virtual devices. Timing and source-frame association must remain consistent through prediction and presentation. Motion is predicted once, as in the ALVR reference client. Poses are sampled on a grid locked to the local display (section 7.3). The pinned server (since ALVR 4a463af5) extrapolates every received pose by its measured motion-to-photon latency with the velocities sent alongside it (controllers by that latency minus SteamVR's own pipeline offset). From RC4.1 the client therefore samples current poses and labels them with the target time `now + measured pipeline latency` (ALVR's estimate, capped by `max_prediction_ms`). RC2-RC4 also sampled SteamVR at that target time, which predicted every pose twice. Frame timestamps are those strictly increasing targets, while freshness, hold windows and frame age use the local acquisition time. The client toggle turns prediction off: poses are timestamped at sampling and, with an RC4.1 driver (patch 0014), the host does not extrapolate them. Controller input, skeletons and proximity follow the streaming session with a connected headset, not momentary head-pose validity or compositor focus. Both hands carry buttons, touches, sticks, trigger/grip input, native skeletal information and haptics. The patched protocol supports Index skeletal data for both WithController and WithoutController motion ranges; hand identity and validity must survive reconnects and transient device loss.

From 1.0.0.6 the client measures what that extrapolation does to the controllers: for every controller sample it extrapolates the sample one prediction step earlier the way the host does (position + velocity × step) and records the distance to where the controller is now (`client/device/prediction_check.hpp`). Each `INPUT` line carries a hand's window (`prediction`: samples, step, mean, 95th percentile, largest, and the largest while both samples were slower than 3 cm/s); the timeline keeps each 10 seconds' largest (`predict_max_mm_l`, `predict_max_mm_r`) and reports the session's (`controller_prediction`). Only distances are kept. A controller that stops suddenly overshoots for one prediction step by its speed times the step (about 45 to 60 ms on the stream, about 20 ms for a headset on its own PC), and settles when held still. In the 1.0.0.5 test controller models held still against each other overlapped about 10 mm more on the stream than on the headset's own PC, which the prediction cannot do. So the client also keeps the distance between the two controllers while both are held still (`pair` in the `INPUT` line: samples, mean, least and most), and the host logs, per 5-second window while both are held still, their distance as received and as submitted to its SteamVR after the prediction and the configured offsets, with the largest move and turn it gave either one (`VROST_CONTROLLER_PAIR received_mm submitted_mm max_shift_mm max_turn_deg`, driver patch 0020, `alvr/server_openvr/src/vrostbridge_pair.rs`). Equal distances mean the poses reach the host's SteamVR as tracked.

Finger fidelity depends on the physical Index sensors and the runtime's own estimation as well as forwarding. Compare a suspect movement locally with the same game and hand before diagnosing transport. A tracking pose alone does not prove that action input is active; acceptance checks both hands independently over extended play and after sleep/reconnect.

The intended menu is the host SteamVR dashboard. The client suppresses its own dashboard and forwards the System action. Left B plus thumbstick click and the desktop Open host menu control provide alternate paths. Menu input must not strand the user in a local SteamVR floor scene.

From 1.0.0.10 the System press is forwarded whole (`client/device/input_protocol.hpp`: `SystemButtonLatch`, `SystemButtonEvents`, `holdThroughFocusLoss`). While the system button is held, the headset PC's own SteamVR takes input focus from the client, ready for its long-press recentre, and without focus every action reads as released, so in the 1.0.0.9 test the host saw a menu press for a few milliseconds or not at all. The client now also reads SteamVR's own press and release events for the system button, which SteamVR is set to send to every application (`sendSystemButtonToAllApps`), focused or not; that they arrive while SteamVR holds focus is still to be confirmed on hardware. A press starts from a press read either way, is forwarded for at least 0.12 s, and ends on a release read either way or, while the button cannot be read, after those 0.12 s, as a tap; a new press needs a release first, so one press is never sent twice; a press event gives way to a reading with focus that says released once it is a

quarter of a second old, and lapses after 10 s. A loss of focus shorter than 0.5 s keeps every other control at its last value, so a short menu press does not drop what the user holds; a longer one (a long press included) releases them as before. The host's SteamVR then treats the press as it would a headset plugged into it: the dashboard on a short press, and its own recentre on a long one whenever the client can follow the press. Each press is logged (`SYSTEM_BUTTON`: how long it was forwarded, whether focus was lost meanwhile, and `unread=1` when for a moment the button could be read neither with focus nor from SteamVR's events, so the press may have reached the host as a tap), and the `INPUT` lines count focus losses, SteamVR's button events and, per hand, the presses, those with focus lost and those unread.

The local SteamVR room setup is authoritative for physical safety and must be completed before room-scale testing. The bridge transfers the supported rectangular play-area size and standing-space transform to the host and verifies application through its helper. It does not clone arbitrary irregular chaperone polygons or replace the physical client's boundary system. Seated diagnostic testing can use an appropriate local standing/seated origin; room-scale release acceptance requires a correct local room setup.

9. Audio and optional eye tracking

Game audio is transported by the reused ALVR stack and has been confirmed working by the user. In the 1.0.0.8 test it lagged, broke up and crackled at times, and the causes were in ALVR's pinned audio code at both ends. From 1.0.0.9 the host captures game audio from Windows' loopback stream itself (patch 0023): packets Windows marks silent are sent as silence, capture gaps are counted, the thread runs in Windows' Pro Audio class, a silent render stream keeps the audio engine running through quiet moments, and surround is folded to stereo by speaker position with a soft limit. ALVR's cpal capture had hidden the packet flags, sent nothing while silent, and folded 5.1 one channel off; it remains the fallback. Both clients play it through VROSTBridge's own player (patch 0022). The device is opened at the stream's rate where offered (cpal's ALSA default had been 44.1 kHz, converted by rodio without a filter), otherwise a windowed-sinc converter is used. The buffer is held at the host's target (30 ms wired, 50 ms Wi-Fi) by a speed correction of at most ±0.5 %, so clock drift never empties or overfills it. A lost packet is cross-faded instead of emptying the buffer, a dry buffer fades and raises the target by 10 ms (up to 150 ms, relaxing after a calm minute), and the device buffer is 40 ms. From RC4 the client makes the headset's own speakers and microphone the default devices for the session when it can match them by name, and restores the previous defaults afterwards. On Windows (1.0.0.7) SteamVR switches playback and recording to the headset itself while it runs, so the client changes no audio device there and only names the current defaults. Microphone forwarding is opt-in. Host preflight checks for an appropriately configured Windows virtual audio endpoint; no virtual audio driver is bundled. Missing microphone prerequisites produce a clear warning without unnecessarily aborting video and game-audio streaming. Windows lists a microphone only when a signed audio driver provides one, so from RC4.3 the host page's Headset microphone card checks the devices and, when the user selects Set up headset microphone, downloads VB-CABLE (VB-Audio Software, donationware) from VB-Audio's own server, accepts it only if it matches the pinned `VBCABLE_Driver_Pack45.zip` SHA-256 or its setup program carries a valid Authenticode signature by VB-Audio's author, and opens VB-Audio's setup program itself with the Windows runas verb, so the UAC prompt names VB-Audio as the verified publisher (never silently: VB-CABLE's licence does not allow building it into another installation). The package is extracted to a new folder and its files are held open (read sharing only) from verification to the end of the setup: they must be unchanged since the download, and a newer package's signer must be exactly CN=BUREL VINCENT Entrepreneur individuel with SERIALNUMBER=423 734 177. The folder is deleted afterwards. The card then reports ready, restart needed or not installed, and the session warning names the missing step.

Beyond 2e eye cameras use the separate bundled go-bsb-cams helper with a loopback-only HTTP listener (`http://127.0.0.1:8880/stream`); from 1.0.0 its MJPEG server is VROSTBridge's own `mjpeg_stream.go` (same multipart stream, each part flushed at once) in place of an unlicensed module. Up to 1.0.0.2 it kept the module's pacing: at most one frame per 50 ms per client, and only the first to arrive

after that pause (17 a second in a 1.0.0.2 report). From 1.0.0.3 each client gets the newest frame it does not have yet as soon as it arrives, at most one every 15 ms. Baballonia's eye filter (One Euro) is set once from its defaults (0.5 Hz, speed 3) to 1 Hz and 10, while it is still at those defaults. Together they cut the time to follow a 5 to 10 degree eye movement from 235 to 350 ms to 70 to 85 ms in a simulation of Baballonia's filter, with no more jitter at rest. From 1.0.0.5 (`jpeg_frames.go`, `camera_stats.go`, the vendored go-uvc changes; `VROSTBRIDGE_CHANGES.md`) the helper passes the cameras' Motion-JPEG pictures on as they were sent: up to 1.0.0.4 it decoded and encoded each again, 10 to 16 ms of processor time per picture on a handheld and a whole core in the 1.0.0.4 test, and pictures queued when that was slower than the cameras. A picture is checked (one JPEG from its start to its end marker with a frame header and a scan; the standard Huffman tables added where the camera leaves them out); the first five after opening and one in 120 are decoded in full, and one Go cannot decode is not sent; one Baballonia's reader would cut short (FF D9 inside a header) or could not hold (over 900 KiB) is encoded again. The camera is asked for its fastest frame interval (upstream committed GET_MAX, the slowest). A picture that lost USB data, carries the UVC error bit or does not fit is skipped and counted; the camera is opened again, after everything opened before is closed, only when no picture has come for 2 s (5 s after opening; also while USB data keeps coming without a picture) or the device is gone, with waits from 250 ms to 4 s, while the HTTP stream stays up. Reads use 100 isochronous transfers of at most 32 packets, and each client gets at most one picture every 8 ms. Every 5 s the helper writes a `STATS` line (sent, received, damaged frames, lost packets, rejected, re-openings, the longest gap, the interval and size) and a `CAMERA` line whenever it opens a camera. Baballonia's filter is set to 2 Hz and 60 where it is still Baballonia's or 1.0.0.3's (90 % of a 5 or 10 degree movement within 33 ms, two camera pictures, instead of 67 to 83 ms). Baballonia (Project Babble, Babble Software Distribution License 1.0, non-commercial) owns inference models and calibration; VROSTBridge accepts its VRChat-style OSC output locally on port 9005. Camera images are not forwarded to the rendering host.

From RC4.5 VROSTBridge sets up and runs these itself (`launcher/baballonia.py`, `launcher/eye_tracking.py`; user guide `docs/BEYOND_EYE_TRACKING.md`):

- **Set-up on request.** After the user accepts Baballonia's licence (named on the page and in Game Mode), the companion downloads the unmodified Linux release 1.1.1.0rc6 from Project Babble's GitHub page with curl (HTTPS only, the desktop's proxy settings, under the watcher below) or urllib.
  - It accepts the download only at the pinned size (517,439,412 bytes) and SHA-256 (`06c9ce7e…906a`).
  - It unpacks it in one pass into a staging folder, refusing absolute, `..` or backslash names and anything but plain files and folders (`filter='data'`), and renames it to `~/.local/share/vrostbridge/baballonia/1.1.1.0rc6` with a receipt.
  - Cancelled or failed set-ups and stale staging folders are removed; Remove Baballonia deletes the folder and keeps `~/.config/ProjectBabble`. Up to 1.0.0.2 uninstall deleted it too; from 1.0.0.3 the uninstaller asks (Keep, the default button; Remove; Cancel), and a kept copy is used by the next installation.
  - 1.0.0.3: the release in use is the pinned one once it is set up; until then one an earlier VROSTBridge set up here (its receipt names its version and SHA-256), which keeps working until the user updates it (Accept licence and update to …). A copy an installation before 1.0.0.1 left under another folder name (`<data>/<name>/baballonia/<pinned version>` with a `*-receipt.json` naming the pinned version and SHA-256) is moved into place at start, without a download. The installer's last message says whether the eye tracking set up before was found and kept.
- **Settings.** Only while Baballonia is closed (it keeps its settings in memory and writes them back), VROSTBridge sets:
  - its two cameras: the helper's stream through its `IpCameraCapture` reader, auto-started;
  - the eye areas once: the halves of a 2:1 picture, as its own select the entire frame; existing areas that fit are kept;
  - its output: VRChat native eye tracking to 127.0.0.1:9005, OSCQuery off under both key names; SteamVR auto-start, update check and GPU off; onboarding once.

  - Atomic writes at mode 0600, a one-time `LocalSettings.json.before-vrostbridge`; set-up state in `~/.local/state/VROSTBridge/baballonia-setup.json`.

- **Owners and the worker.** The session (Start with Eye tracking on, until Stop) and the user (Open Baballonia, until Close Baballonia or its window closes) want eye tracking. One worker thread reconciles that every 2 s or when told, so Start and Stop never wait for it.
  - **Order:** the relay opens first (Baballonia's OSC sender stops for good after one refused send) and closes only after Baballonia stops. The camera helper starts next, and the companion waits for its first JPEG (`probe_frame_size`, which also gives the eye areas their size). Baballonia comes last.
  - **Process ownership:** Baballonia runs as the user unit `vrostbridge-baballonia` (systemd-run, `KillMode=mixed`, `TimeoutStopSec=6`; a direct process group without systemd), outside Steam's app scope so gamescope never focuses it.
  - **The watcher:** Baballonia, the camera helper and the download run under a POSIX shell watcher (`WATCH_SCRIPT`, free of `${` and `%` for systemd). It ends its program when the companion's pid is gone or a zombie, and stops it with SIGTERM then SIGKILL after the grace (3 s for Baballonia, which ignores SIGTERM).
  - **Recovery:**
  - A camera helper that ends makes the worker stop Baballonia at once (its MJPEG reader never reconnects and spins a core once the stream ends) and start both again; during a calibration it is kept and restarted when the calibration is saved.
  - 25 s without a sample while both run restarts both, at most once in 5 minutes, never during a calibration or the 15 s after it (until its new model path is saved).
  - A Baballonia started before this companion is restarted; one that closes is left closed (Open Baballonia).
  - Another Baballonia install blocks the start with its path.
  - When nobody wants eye tracking any more, a running calibration may finish (up to 15 minutes).
- **Calibration (1.0.0).** Baballonia decides where to show a calibration when one starts (`OverlayProgram`): when a process named like `vrserver`, `monado` or `wivrn-server` runs it starts its Godot calibration program with OpenXR (`--xr-mode on`), in the headset; otherwise in a window on the screen (`--use-debug`). So a calibration reaches the headset only during a session, while SteamVR runs on the client.
  - With OpenXR the calibration is a new SteamVR scene application, and SteamVR sends the application showing the stream `VREvent_Quit` ("Local SteamVR requested this client to quit"): in the first test that ended the session, then SteamVR, then Baballonia. So VROSTBridge replaces the file Baballonia starts, `Calibration/Linux/Overlay/BabbleCalibration.x86_64`, with a POSIX shell launcher (`CALIBRATION_LAUNCHER` in `launcher/baballonia.py`, MIT, no Baballonia code) and keeps Baballonia's program unchanged beside it as `BabbleCalibration.program` (Godot finds `BabbleCalibration.pck` from the executable's name). With `vrserver` running and `--xr-mode` among the arguments, the launcher drops `--xr-mode` (and its value) and `--use-openxr-overlay` and adds Baballonia's own `--use-openvr`: the calibration runs as a SteamVR overlay (godot_openvr), as Baballonia does on Windows, over the stream, which keeps running. Otherwise the arguments pass unchanged. `prepare_calibration` puts the launcher in place before each start (atomic write; the program is restored if the write fails); Remove Baballonia deletes both. `VROSTBRIDGE_CALIBRATION=openxr` in the companion's environment keeps Baballonia's OpenXR way.
  - **The overlay's Vulkan layer (1.0.0.2).** SteamVR's `vrclient.so` takes an overlay's Vulkan texture through extension commands on the program's own `VkDevice` (external memory and semaphore file descriptors, timeline semaphores). Godot 4.5 enables `VK_KHR_external_memory_fd` but not `VK_KHR_external_semaphore_fd`; Mesa returns NULL for commands of extensions not enabled, and `vrclient.so` called it: in the 1.0.0.1 test the calibration crashed (SIGSEGV at address 0 inside `IVROverlay::SetOverlayTexture`) as soon as it showed its first picture, in a session and with SteamVR alone, and Baballonia kept waiting for it. VROSTBridge's own layer (`client/platform/linux/openvr_texture_layer.c`, `VK_LAYER_VROSTBRIDGE_openvr_textures`, libc only,

MIT) adds at `vkCreateInstance` the external-capability instance extensions and at `vkCreateDevice` the external memory, semaphore and fence extensions with their fd forms and what they build on, dma-buf, the foreign queue family, DRM format modifiers, timeline semaphores and synchronization2 (with their features where supported and not set by the program), each only where the driver offers it and the program did not ask for it; if the driver refuses the longer list the program's own is used. It writes one stderr line per device (Baballonia's log): what it added and any command SteamVR calls that is still missing. The launcher enables it for `--use-openvr` only (`VK_ADD_LAYER_PATH` = `vrostbridge-vulkan` beside it, `VK_INSTANCE_LAYERS`, both ahead of the user's own), when its manifest is there and `VROSTBRIDGE_VULKAN_LAYER` is not `off`; `prepare_vulkan_layer` copies it there from the installation's `vulkan/` whenever it differs (manifest last; a failed copy is logged and the calibration starts without it).

  - For that OpenXR way VROSTBridge starts Baballonia with `XR_RUNTIME_JSON` set to SteamVR's `steamxr_linux64.json` in the runtime the OpenVR registry lists first, so it uses SteamVR whichever OpenXR runtime the PC has active (SteamVR's own button to set itself often does nothing on Linux). Only then: the start's log line says which, and `baballonia.steamvr_openxr` on the status says whether that file exists (read at most every 10 s; `null` otherwise, so the page never warns about a file the overlay does not use). Without the opt-out no OpenXR runtime is forced, so with Monado or WiVRn Baballonia's OpenXR calibration keeps the PC's own runtime.
  - Stopping the session during training ends SteamVR on the client, which closes Baballonia (it holds an OpenVR background connection), so the calibrating message says to keep the session running until it has finished.
  - For a session in Desktop Mode (not Game Mode, not Open Baballonia), the worker minimises Baballonia's window once it appears (`launcher/x11_window.py`: XWayland/X11 `_NET_CLIENT_LIST`, `_NET_WM_PID`, `WM_CLASS` `Baballonia.Desktop`, `XIconifyWindow`; up to 60 s), so it does not cover the page; the `eye-show` action (Show Baballonia) activates it again (`_NET_ACTIVE_WINDOW`).
  - While Baballonia runs, the worker checks the same process names (`xr_runtime_running`) and reports `eye_tracking.steamvr`; without one, the running and calibrating messages say a calibration shows on this screen and to start a session first. Without a session (SteamVR started by hand) they say to calibrate during a session: in the first test, Baballonia's OpenXR calibration showed each eye the other eye's picture with SteamVR started by hand (SteamVR Home running) and was right during a VROSTBridge session (Home and the dashboard off). The card and Game Mode name Baballonia's routines: Eye first (gaze and blinks, then training), Eye (no tutorial) later, Gaze and Blink redo one half after a full one (each merges with the other half's earlier capture).

- **The session's choice.** The sharp region follows the eyes only when, at Start, Eye tracking is on, the headset is a Beyond (or not yet known), and Baballonia is set up and calibrated (`EyeHome_EyeModel` an absolute `tuned_temporal_eye_tracking_*.onnx` that exists).
  - The choice is made again when the detected headset is of another family than the one Start assumed (`eye_for_headset`; the host is then restarted with it, because the session settings carry the headset), and eye tracking ends for a headset without eye cameras. A calibration saved while a session starts applies from the next Start, so the host started early and this PC never disagree.
  - Otherwise Eye-tracked falls back to Balanced while Baballonia runs so it can be calibrated. In Game Mode, an uncalibrated Baballonia is not started: its window, needed to calibrate, cannot be shown there.
- **Camera permission.** Allow eye camera access runs one fixed `/bin/sh` script through pkexec, in the background. The script writes `/etc/udev/rules.d/70-vrostbridge-bigeye.rules` (passed as its argument), reloads udev rules and triggers a change event for 35bd:0202 so systemd's `uaccess` applies.
- **Priority (1.0.0.4).** Baballonia starts under `nice -n 5` where `/usr/bin/nice` exists: with spare processor time it runs as fast as before, and when the processor is short the stream's own work goes first.
- **Status.** The companion's `eye_tracking` status (phase, message, owners, Baballonia state and calibration date, cameras, relay counts and rate, camera-access prompt; from 1.0.0.4 the seconds

since the last sample and the session's pauses between samples, counted over 100 ms, 250 ms, 1 s and 5 s with the longest) feeds the page's Eye tracking card, Game Mode's This PC → Eye tracking page and the `eye_state` notes. From 1.0.0.4 the eye-tracking view says that the sharp region holds where the user last looked while no sample arrives, and warns when eye tracking sends fewer than 30 samples a second. From 1.0.0.5 the status adds the camera helper's latest `STATS` values (`cameras.feed`, fresh for 15 s) and its `CAMERA` lines, and the card says how many camera pictures a second the samples come from.

  – From 1.0.0.6 the relay also counts each sample's yaw and pitch (`gaze_provider.GazeReach`: counts per whole degree, reset each session or by Measure again), and the status's `relay.reach` gives how far the gaze went each way (the 1st and 99th percentile of each axis, after 100 samples). The card shows it as Gaze reach this session: … left · … right · … up · … down (Baballonia reports up to 45°), with a hint to calibrate again turning the head further when a way stays under 30°; Game Mode's Eye tracking page has it as a row that measures again. Baballonia's model reports only as far as its calibration went (the calibration's dot stays still while the head turns, so the eyes go only as far as the head turned) and never more than 45° from straight ahead (its trainer's label range); in the 1.0.0.5 test the region followed the eyes only near the middle of the view.

  – Reports add that status and the end of `baballonia.log`, never gaze samples: only the session's reach (four whole-degree numbers) and the host's per-window extremes.

From 1.0.0.7 the Windows client runs the same eye tracking (the switch, the licence, one calibration, the owners and worker, the relay, the recovery and the status), with these differences (`launcher/baballonia.py`, `launcher/eye_tracking.py`, `launcher/gaze_provider.py`):

- **The cameras.** Windows lists the Beyond 2e's eye cameras (USB 35bd:0202, one 800 × 400 picture with both eyes side by side) as an ordinary camera named `BigEye`, and Baballonia reads it itself through DirectShow (its `OpenCvCapture` source, both eyes from that one picture): no camera helper, no port 8880, no udev rule and no camera-access prompt. A camera has one reader at a time, so Bigscreen's own eye-tracking app must be closed while Baballonia runs. The companion checks that Windows lists the camera (WMI; a camera Windows names USB Video Device instead is used under that name, with Bigscreen's advice to let Windows name it again) and, while eye tracking runs, every 3 seconds that it is still connected: when it goes, Baballonia is stopped (its reader would spin a core, as on Linux) and both start again once Windows lists it.
- **Set-up.** The pin is Project Babble's Windows installer of the same release, `Baballonia.Setup.v1.1.1.0rc6.exe`: 423,324,292 bytes, SHA-256 `dec2e8fd1e5c844662f99b47680d772a393f6eb2fdf4e4dd3c62853928b2c470`, downloaded only after the licence is accepted and checked before it runs. It runs silently for this user into `%LOCALAPPDATA%\VROSTBridge\Baballonia\<version>` (877 MB installed), below normal priority, in a job object that lets it start no other program (its installer would start Baballonia when it finishes); three of the program files it installs are checked by SHA-256, and the uninstall entry and Start menu and desktop shortcuts it writes are put back as they were, so VROSTBridge's copy stays private. A Baballonia the user installed is used instead when its version (where Windows records one) and those three files match the pin: it runs from where it is, nothing is downloaded, and Remove Baballonia only stops using it.
- **Settings.** Baballonia's own folder is `%APPDATA%\ProjectBabble`; VROSTBridge writes the same output, filter and eye areas, with both cameras set to the `BigEye` camera (`OpenCvCapture`). Baballonia's Windows default runs its eye model on the GPU (`AppSettings_UseGPU`); VROSTBridge sets the processor once, as Linux runs it, and keeps any later choice made in Baballonia.
- **Running.** Baballonia starts minimised without taking the focus, below normal priority, in a job object that ends it, its calibration overlay and its trainer with the companion. Its window is minimised and shown again (Show Baballonia) through Win32. Stopping closes its window, so it saves its settings, and ends it after 5 seconds. Its own log, in `Documents\ProjectBabble`, goes into reports.

- **Calibration.** With SteamVR running, Baballonia's Windows release starts its calibration as a SteamVR overlay by itself, so it shows over the stream: calibrate once in Baballonia's window during a session, after closing Bigscreen's app. The calibration launcher, the Vulkan layer and the OpenXR runtime check are Linux's and are not used.
- **The gaze** reaches the native client on a loopback UDP port (section 4.4) instead of the Unix socket.
- **Not yet seen on hardware:** the silent installation and its job object, Baballonia reading `BigEye`, and its calibration overlay over the stream.

The relay accepts the documented centre or binocular pitch/yaw messages, rejects malformed/nonfinite or scheduled data, forms a normalized combined orientation and forwards fresh samples. Independent per-eye gaze/vergence, confidence and eyelid state are not carried. The native 50 ms freshness cutoff starts at local receipt, not camera exposure, so it cannot prove that a provider's image estimate is current. Missing gaze is not a blink.

The implemented host application paths are calibrated combined gaze for supported OSC use such as VRChat (from RC4.5 turned on for a client's session by the `session-gaze` feature) and, from RC4, gaze-driven foveated encoding (section 7). No host OpenXR gaze extension or dynamic foveated rendering for games is claimed. Physical acceptance must verify calibration after refitting, direction, stale-data handling and camera/provider restart. Ordinary video/tracking must remain usable when gaze is disabled or unavailable.

10. Network and trust model

Endpoint	Purpose	Scope
UDP 9946	Companion host discovery and replies	Eligible private LAN interfaces
TCP 9946	Pair/start/heartbeat/stop/status broker	Restricted session API with bearer credentials
TCP 9943 / UDP 9944	Native ALVR connection and stream	Selected host/client network path
UDP 5353	Native service discovery	Local network
TCP 9945	Read-only client diagnostics	Recognized peer; bounded post-disconnect grace
Local desktop HTTP (TCP 9947 client, 9948 host, from 1.0.0; any free port if taken)	Browser UI and authenticated local controls	Loopback, not a general LAN web service
UDP 9 / 7 (outbound broadcast)	Wake-on-LAN magic packets from the client	Client's LAN broadcast and the last host address
UDP 9944 (between sessions)	RC4 network test from the host to the requesting client	Paired client that asked; never during a session
TCP 8880 / UDP 9005 (client)	RC4.5 eye cameras' MJPEG stream and Baballonia's OSC output to the gaze relay	127.0.0.1 only, while eye tracking runs; other local programs of the same PC can reach them (no 8880 on Windows, where Baballonia reads the camera itself)
UDP, a free port per session (Windows client)	1.0.0.7: the gaze relay to the native client (`--gaze-udp`), in place of Bazzite's Unix socket	127.0.0.1 only, bound by the native client for its exclusive use

From RC4.2 the client keeps a Wi-Fi session on one access point: it sets `802-11-wireless.bssid` on the active connection in memory (`nmcli connection modify --temporary`, so a restart also undoes it) and reconnects once at Start, because NetworkManager runs no background scans for a BSSID-locked connection; scans took the radio off channel every 300 s with several access points. A receipt restores the profile at Stop or the next start and is removed only once the lock is known to be gone. The lock is skipped when only the current access point is known with a good signal and personal security, on Wi-Fi 7 multi-link, or when the profile already names a BSSID. The client also reports whether the stream uses Wi-Fi or Ethernet, with link warnings. From RC2 the host profile enables ALVR's DSCP marking (Expedited Forwarding); on Linux this prioritises the client's uplink on Wi-Fi, while Windows ignores application marks without a QoS policy. During a session on Wi-Fi the client asks NetworkManager to disable power saving for that adapter with a runtime-only change that is restored at Stop; NetworkManager usually refuses that change for a running connection, so RC4 offers a user-initiated permanent change to the home connection's profile. An optional host QoS policy (RC4) marks SteamVR's video packets for Wi-Fi priority after administrator approval. Host firewall rules may include both TCP and UDP variants of the native ALVR ports. The Bazzite firewall helper opens its explicitly defined inbound set on the observed home subnet. The Windows client (1.0.0.7) opens exactly the same ports and no more: `launcher/Client-Firewall.ps1`, run by Allow home-network connection after one UAC prompt, creates five inbound rules named `VROSTBridge.Client.*`, Private profile and local subnet only, each for the program that listens (TCP 9943, UDP 9944 and UDP 5353 for `VROSTBridgeClient.exe`; TCP 9945 and UDP 9944 and 9946 for the companion's `python.exe`), with an ownership marker, so they never touch the host's rules on a PC that has both; the uninstaller removes them after one more prompt. Its check also reports adapters on a network Windows classes as Public, where the rules do not apply, and block rules that a declined Windows Firewall prompt left for either program, which win over allow rules. Configuration is scoped; no installer disables a firewall or broadens the user's network category. First-time firewall setup can request operating-system administrator consent.

On a Windows headset PC (1.0.0.7) the Wi-Fi session goes through Windows' Native Wifi API instead of NetworkManager: the adapter runs in media streaming mode and with background scanning off for the session, which is what the Bazzite access-point lock achieves (no reconnect and no profile change are needed), and other Wi-Fi adapters on the same network are disconnected and reconnected at Stop, with a receipt. Both settings last while VROSTBridge's Wi-Fi handle is open, so they also end if the companion exits, and Windows drops them when Wi-Fi disconnects, so they are set again after a reconnect. Wi-Fi power saving is a power-plan setting on Windows; Turn off Wi-Fi power saving sets the plan's wireless setting to Maximum Performance, on battery and plugged in. Windows 11 24H2 and later name the network and its access point only to desktop apps with location permission: without it the page says where to allow it, roaming detection and the pausing of other adapters are off, and streaming is unaffected.

The companion protocol is `vrostbridge-session-2`. From RC3 a host advertises `capabilities` (`quality-2` for the encoder/network/timing-lock session fields, `pipeline-1` for the heartbeat pipeline summary, `vsync-lock-1` when its driver includes patch 0007); RC4 adds `quality-3` for the 10-bit, AV1, foveation, pacing, intra-refresh and quick-resume fields, a separate `features` list derived from the driver manifest (`pacing`, `gaze-foveation`, `optical-centre`, `intra-refresh`, `link-test`, ...), the authenticated `/linktest` action and a small `host_info` status block (wake address of the adapter the client reached, power state, quick-resume time); RC4.1 adds `quality-4`; RC4.2 adds the `report-upload` feature and the authenticated `/report` action (a client report as base64 gzip JSON: at most 6 MiB in the request and 16 MiB decoded, read only after the paired credentials in its headers are checked, one at a time and one every 3 seconds, subject to the same network policy as Start; the host stores the decoded JSON unchanged, at most 20 reports and 96 MB, under names it numbers itself); RC4.5 adds the `session-gaze` feature: such a host accepts a boolean `gaze` session field and turns ALVR's face tracking (the VRChat eye OSC sink) on for that session, remembering in its state folder (`host-session-gaze.json`) that `session.json`'s value is session-only so it is never read back as the host's own preference, and quick resume restarts SteamVR when it changes; clients send the newer fields only to hosts that advertise them, so mixed versions keep working; the native marker is `vrostbridge-native-1`, with pinned ALVR wire identity

`21-dev14-vrostbridge-native-1`. Protocol identity is checked independently of display version labels. The broker bounds request sizes (8192 bytes), validates settings and exposes no arbitrary shell command, URL fetch or filesystem path execution. Pairing stores a per-client credential and a host-side hash; status output does not return credentials.

Local web mutations are guarded by authentication tokens and Origin/Host checks. Client diagnostics are read-only and peer-restricted, with a 60-second collection grace after disconnect. These measures are not a claim of end-to-end encrypted streaming or safety on an untrusted LAN. Do not port-forward the service, treat private addressing as user identity, or present the product as an Internet service. Authentication and network scope are distinct from transport encryption.

11. User interfaces and Game Mode

Desktop use retains a browser interface on each computer. The host's copy describes rendering on this PC and a headset on the remote client; the client's copy describes the headset here and rendering on the host. The shared VROSTBridge icon and branding identify both roles, with the role in the name (VROSTBridge Host, VROSTBridge Client) rather than in a second icon or colour. From 1.0.0 the pages use the brand palette (cyan #00E5FF, blue #0085FF, purple #A855FF, magenta #FF2DCB), the symbol in the header beside the name (from 1.0.0.3 in the logo's lettering on one line: VROST, then Bridge with a capital B, set in Montserrat and kept as outlines) with VR Rendering OutSide The Box - Bridge under it (what the name stands for, from 1.0.0.1 in full: V, R of Rendering, O, S, T and B of Box in the palette's colours, every letter of Bridge in the brand gradient; the badge beside it names the role, and the Game Mode screen has the same line), the logo with the descriptor, version, licence and independent-project notice in About, and the brand favicons. Controls prioritise connection state, Start/Stop on the client, readable quality settings, headset detection and Save report. Advanced options and credits remain available without dominating the primary flow.

The Windows tray supports a visible right-click menu, Settings, Exit and background status. Its icon is the one-colour symbol (fitted to the pixel grid at 16 to 32 px), white on a dark taskbar and dark on a light one (read from `SystemUsesLightTheme` and swapped when Windows reports a theme change); the tooltip names VROSTBridge Host and its state. Launch-at-sign-in is configurable and does not open a browser. A second ordinary launch reopens the existing UI rather than creating a duplicate companion. From 1.0.0 that leaves one page tab: each page announces itself on a `BroadcastChannel` (`vrostbridge-page`) when it opens, and an older page closes itself (or, where the browser does not allow that, replaces its content with a note) when a newer one opens, on both PCs (Windows tray Open VROSTBridge, Bazzite taskbar pin). A browser cannot be asked to switch to an existing tab from outside, so the newest tab is the one kept. The pages are served on a stable loopback port per role when it is free (9947 client, 9948 host; otherwise any free port, as before), so a page from an earlier companion run is the same origin and is closed too. Startup logs distinguish native launcher failures from Python/UI and session failures.

The Windows client (1.0.0.7) has a tray icon of its own, named VROSTBridge Client with the session's state, whose menu offers Open VROSTBridge, Start session, Stop session (whichever applies) and Quit; a session started there reports its first picture, or why it stopped or did not start, as a Windows notification, because no page may be open. Start VROSTBridge Client when I sign in is a switch on the page and a choice in the installer. The page is the Bazzite client's, with the same actions; with `platform: windows` in the status it shows the Windows wording and controls (the sign-in switch, the power mode, media streaming mode and the location note) and hides what is Bazzite's (Game Mode, Add to Steam, Fix SteamVR reprojection, eye-camera access, the GPU clock fix). Those actions, if called, say why they do not apply on Windows. A long message, such as one naming a Windows path, wraps instead of widening the page (both clients).

In the headset, SteamVR-style status cards (logo, Inter/Outfit) and the quality pill are head-locked OpenVR overlays sent as raw images (`SetOverlayRaw`, at most 2 MiB per image). From RC4.4 they are drawn at up to twice the pixels per degree (`StatusCardRenderer::card/pill` with a scale, `ui::fittedScale` keeping each image within 2 MiB: the pill at 2×, a large reconnecting card at about 1.35×) and the

overlay width is divided by the same scale, so SteamVR shrinks them instead of enlarging them; each image is drawn once per change, starting from the scale the last image of its kind allowed; RC4.3 drew them at about one image pixel per headset pixel and the lens correction enlarged them.

Game Mode is Bazzite's (gamescope's full-screen Steam session there); the Windows client has none and does not need it, since SteamVR on Windows drives the headset itself and the client lives in the tray (section 4.4). Game Mode uses a native SDL interface, not a browser tab or Proton. It shares preferences and session logic with desktop mode. From RC4.2 it is drawn at the display's native resolution with the pages' fonts (Inter/Outfit rasterised by `libvrosttext.so`) and colours. D-pad/left stick or keyboard navigation (focus moves to the nearest control in that direction), A/Enter selection, B/Escape, mouse and touch are supported. From 1.0.0.4 (`launcher/game_ui.py`): on a settings list, left and right change the focused row's value (the previous or next value; switches off with left, on with right) instead of moving focus, up and down step through the rows, rows scrolled out of view are reached only by stepping through the list, and a list stops at its ends (other pages still wrap); a held D-pad direction or arrow key repeats after 0.38 s, then every 0.11 s; the stick moves once past about 55 % of its travel, repeats every 0.14 s while held and releases below about a third; one press reported by two sources within 70 ms (Steam's virtual controller and the physical one, or a button and a key) moves once; controllers are opened once and kept, so a lost release cannot leave a direction repeating; the hint line shows ◀ ▶ Change on settings lists. From RC4.3 it has every client setting of the desktop page on scrolling pages (Stream & headset, Motion & timing, Network & host, This PC; RC4.5 adds This PC → Eye tracking: the switch, set-up with the licence, progress, Open/Close Baballonia, camera access and Remove, also reached from the eye-tracking note on Stream & headset), the focused row explains itself, B/Escape go up one level and do nothing on the main page, and Start stays available: when the headset display is not reserved the session card gives the reason. Add to Steam on the client page (Desktop Mode) closes Steam, adds a non-Steam shortcut named VROSTBridge with library artwork to the most recent account's `shortcuts.vdf` (byte-exact round trip, backup, receipt, other entries unchanged), sets up the integration below and starts Steam again; Remove from Steam and uninstall reverse it.

Bazzite's session passes `--prefer-output *,eDP-1`, so a connected headset became the Game Mode screen and the built-in display went dark. From RC4.2 the wrapper rewrites the list so connected headset outputs (by EDID) and remembered headset ports that are empty now come after every other output; a remembered port with an ordinary monitor keeps its place. Any failure in the wrapper's additions, or a VROSTBridge executable that no longer exists, runs plain gamescope with the original arguments.

The integration requires a gamescope build exposing `--lease-connector` and a gamescope-session-plus environment supporting `GAMESCOPE_BIN`. The lease helper sets `PR_SET_PDEATHSIG` so it never outlives the companion; Linux delivers that signal when the thread that started the helper exits, so from RC4.4 the companion starts it from one long-lived thread (`launcher/game_mode.py PROCESS_STARTER`) instead of the Start request's thread, which would have ended the helper, and the headset display, as soon as Start returned. Its per-user wrapper/drop-in reserves a recognized HMD connector when entering Game Mode. The wrapper records its process and start time and then becomes gamescope; before each Start the companion checks that this gamescope still runs (same process and start time, the user's real and effective UID, gamescope with the headset's `--lease-connector`, and the program link where it can be read). 1.0.0.3: Bazzite's gamescope carries `cap_sys_nice=eip`, which makes it non-dumpable, so even its own user may not read `/proc/PID/exe`. Up to 1.0.0.2 the check required that link and always called the reservation stale. The headset must be connected and powered beforehand. A helper validates the connector/card/EDID and presents a private Wayland DRM-lease socket for SteamVR. Gamescope retains ownership of the original lease; closing a duplicated descriptor is not claimed to revoke that original lease.

Only head-mounted displays are accepted (RC4.3, `core/headset_edid.hpp` mirrored in `launcher/game_mode.py`): the Valve Index product codes, Beyond BIG/0x1234, and any EDID with the kernel's non-desktop markers (a Microsoft HMD vendor-specific block, version 1/2 or 3 without desktop use, or a DisplayID 2.0 head-mounted VR/AR primary use). RC4.2's fixed codes missed the Beyond 2e. Ordinary displays are refused. When a connected DisplayPort display is not settled by its EDID alone, the

companion also reads the kernel's `non-desktop` connector property through the lease helper (`--list-connectors`, which gives up DRM master at once so a compositor starting at that moment is never refused it), refreshed in the background every 10 s. If any part of the wrapper fails, Game Mode starts plain gamescope with the original library path. The wrapper records each Game Mode start (`gamescope-wrapper.json`: reserved output or reason, connected displays with maker, product and markers) for the page and reports. One runtime lease is managed at a time; SteamVR must stop before the lease adapter. Setup records ownership and does not overwrite user-edited integration files. Disabling removes only owned integration; re-entering Game Mode may be required to release its startup reservation. Physical Game Mode acceptance remains outstanding.

In the 1.0.0.9 test SteamVR started in Game Mode, but its compositor failed six times with error 496 (`VRInitError_Compositor_CannotDRMLeaseDisplay`: the display it reached cannot lend the headset), while the same SteamVR took the headset from the desktop in Desktop Mode at once; the report could not show whether it had reached the adapter's display at all. From 1.0.0.10:

- **A plain display name.** The adapter's socket is `vrostbridge-headset-<pid>` in the runtime directory (`WAYLAND_DISPLAY` set to that name), the usual form, which libraries and container runtimes that accept only a name can find, where it was a path to a socket in a folder of its own. The companion checks the socket (owner, mode, connector) before using it.
- **One Wayland display.** SteamVR starts without `GAMESCOPE_WAYLAND_DISPLAY`, gamescope's own display marker, which offered SteamVR a display that cannot lend the headset, so the only Wayland display it is given is the adapter's (it keeps gamescope's X display, `DISPLAY`, for its windows).
- **A failure explained at once.** When the compositor logs that it cannot lease the display, Start stops within seconds with the reason, since waiting cannot help; in Desktop Mode too, on a desktop that cannot lend the headset.
- **Each step on record.** The adapter logs every program that connects to it and binds the lease interface (`LEASE_CLIENT`), as well as the lease it grants (`LEASE_GRANTED`, as before); the companion logs the display SteamVR's server was actually started with (read from its process environment) and the compositor's own lines about the headset display. The lines of the last Game Mode Start are kept in `game-mode-last-start.json` and go into every client report (`game_mode_last_start`), so a report saved later in Desktop Mode still has them.

12. Installation, upgrades and stored state

The Windows host release is a per-user NSIS installer plus a portable host package. It bundles its launcher runtime, patched host and notices. SteamVR, GPU drivers and the Microsoft runtime prerequisites remain distinct; the installer can offer the Microsoft redistributable through its normal installer UI. Runtime packages are not code-signed.

The Bazzite release is an extracted ZIP containing a desktop installer and payload. It installs under the user's data directory (normally `~/.local/share/vrostbridge`, respecting XDG conventions), without modifying the immutable OS image, layering RPMs, using Proton or requiring a compiler. The native target is x86-64 Linux with glibc 2.39 or later and functioning graphics/audio libraries. Rootless application installation does not remove the need for an optional privileged firewall/udev action.

The Windows client (1.0.0.7) is a per-user NSIS installer, `VROSTBridge-Client-Setup-<version>.exe` (about 31 MB), and a portable ZIP with the same files, `VROSTBridge-Client-Portable-<version>.zip` (`packaging/build_windows_client.py`, `packaging/windows-client-installer.nsi`). The contract:

- **Per user, no administrator rights:** it installs to `%LOCALAPPDATA%\VROSTBridge\Client\<version>` (about 63 MB) and refuses 32-bit Windows and anything older than Windows 10. Like the host's, it is not code-signed, so SmartScreen asks once.

- **Beside the host on one PC:** its own folder, uninstall key `VROSTBridgeClient` (VROSTBridge Client in Settings → Apps), sign-in value and Start menu names: VROSTBridge Client, Client startup logs, Client open-source credits and Uninstall VROSTBridge Client in the shared VROSTBridge folder, which goes only when it is empty.
- **Start when I sign in:** an installer choice, off unless chosen (`/STARTUP` when silent): the Run value `VROSTBridgeClient` in `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, `"…\VROSTBridge Client.exe" --background`. A repair or update keeps it on only when the entry is the client's own command, and never replaces another program's.
- **Install and update:** the staged launcher's `--self-test` checks every file against `SHA256SUMS` before anything is replaced; then `launcher/install_maintenance.py`, run with the package's own Python, closes an open client companion, copies the verified files (a same-version repair is put back if the copy fails), and only then removes older versions.
- **What it carries:** the launcher, the native programs and their DLLs (the transport, FFmpeg's three libraries with their unmodified source and Windows build notes, as the LGPL asks, and `openvr_api.dll`), the companion (`launcher/` without Game Mode's artwork, and the Python modules of `host/` it imports), the embedded CPython taken byte for byte from the verified host payload, `docs/`, `licenses/`, `LICENSE`, `NOTICE` and `THIRD_PARTY.md`. No Microsoft Visual C++ redistributable: the native programs link the C runtime statically.
- **Uninstall** closes the companion, restores SteamVR settings still held by a receipt, removes the client's own firewall rules (one administrator prompt, only if any exist; if it is declined, the rules and the client's files stay and the uninstall stops), and asks whether to keep the downloaded Baballonia (keeping it is the default; `/REMOVEEYETRACKING` removes it when silent). It removes the program files, its shortcuts and uninstall entry, and the sign-in value only when it is the client's own command. It keeps settings, pairing and logs in `%LOCALAPPDATA%\VROSTBridge`, Baballonia's own settings and calibration in `%APPDATA%\ProjectBabble`, and a Baballonia the user installed.

Upgrades verify the staged payload and close the old companion through its authenticated local control path before switching active managed versions. Repair reinstalls damaged owned files. Old files are removed only when identifiable as VROSTBridge's own (including VROSTBridge versions). Preferences, pairing/client identity, reports and retained host configuration are preserved. A blocking runtime or unknown modified installation must produce an actionable failure rather than deleting unrelated data.

From 1.0.0.8 the client's state folder also holds `session-tuning.json` (Bazzite: what the performance balance changed, for restoring after a crash), `base-stations.json` (the stations found and the last command) and `gaze-centre.json` (the gaze correction per calibration), and from 1.0.0.10 `game-mode-last-start.json` (Bazzite: the log lines of the last Start in Game Mode). Windows records recognized upgrades and backups beneath `%LOCALAPPDATA%\VROSTBridge`. Bazzite state normally resides beneath `~/.local/state/VROSTBridge`. Uninstall removes owned program files, shortcuts and integrations while retaining user state where documented. From RC4.5 that includes the Baballonia VROSTBridge downloaded (`~/.local/share/vrostbridge/baballonia`, its user unit stopped first), while Baballonia's own settings and calibration (`~/.config/ProjectBabble`) and the root-owned camera rule (`/etc/udev/rules.d/70-vrostbridge-bigeye.rules`) stay. The Baballonia download survives VROSTBridge updates. On Windows (1.0.0.7) it lives in `%LOCALAPPDATA%\VROSTBridge\Baballonia`, outside the version folders, so it survives updates too. SteamVR, unrelated ALVR installations, other gaze providers and manually edited user files are not wholesale deleted. Optional full state removal is a separate deliberate user action.

13. Diagnostics and support

Both ends retain bounded logs, startup milestones, negotiated settings and incidents. The host can collect client diagnostics while connected and preserve a snapshot for its exported report, reducing the need to retrieve two computers' logs. End-of-session collection is best effort; an unreachable or crashed client cannot supply new data. Save the client report separately if the host snapshot is missing or stale.

From RC4.2 every report starts with `report_info` (which end saved it, version, local and UTC time) and its file name carries the same (`VROSTBridge-Client-report_<version>_<date>_<time>.json`; a host keeps an uploaded report under the name it was sent with). The client can send its report to the host, which lists the latest ones for download (from 1.0.0.9 it keeps the 3 newest). Host reports omit ALVR's per-frame `[GRAPH]` lines and per-event tracking/button/haptics lines from the driver log and keep its latest 120 `[STATS]` lines. A Windows PC may have both the host and the Windows client (1.0.0.7) installed, sharing `%LOCALAPPDATA%\VROSTBridge`, so the client's log there is `client-latest.log` (the host's stays `latest.log`); a Windows client report says which platform it came from and carries Windows' crash records (section 5.3) and Baballonia's own log.

Reports should establish the software/protocol versions, physical HMD identity, requested and actual stream format, effective refresh, decode/presentation state, controller validity, network path, startup/stop history and relevant errors. RC1.3 adds static optical geometry so projection faults can be investigated without guessing FOV. From RC4.1 host reports include the driver's own log (`session_log.txt` and the previous SteamVR start's `session_log.previous.txt`), ALVR connection-state changes and stream recoveries; client reports include `STARTUP` milestones and companion heartbeat failures. From 1.0.0.5 a client report also carries the camera helper's `STATS` and `CAMERA` values, head positions sent and missing (`head_poses`, `head_poses_missing`, `head_untracked_ms`, `head_tracking_result`), the stream's colour and shown size, and the network test's steps with its saved result, and its timeline adds camera pictures a second, losses and re-openings; the host report carries the driver's `VROST_COLOR` and `VROST_NO_HEAD_POSE` lines. From 1.0.0.4 a client report also carries `timeline` (`launcher/timeline.py`): one row per 10 seconds of the session, at most 360 (beyond that the first half hour and the latest), with frame rate (average and lowest), lost frames, keyframe requests, frames skipped until a keyframe, presentation incidents, grey frames, the oldest frame shown, eye-tracking samples a second, the host's frames on the gaze, held and centred, the longest handoff, packet interval and conversion, the processor's and GPU's clocks and load, power, temperature, the cadence classes and the processor time of the session's own programs (`launcher/performance_monitor.py` `PROCESS_GROUPS`, the user's own processes: SteamVR, Steam, the compositor, the audio server, Baballonia, the eye cameras, VROSTBridge; any other program only summed as other, never named). A report is diagnostic evidence, not a benchmark certificate.

The implementation avoids dumping live eye/world pose streams (from RC4.3 the live eye view's `FOVEA` samples are never logged and host reports drop the driver's foveation centres; from 1.0.0.6 the driver logs those centres only while they do not follow the gaze) and masks relevant credentials, but reports can still contain hostnames, IP addresses, device identities, file paths and hardware details. Review/redact before public issue uploads. Include what the user saw, local-versus-bridge comparison, test duration and whether the runtime restarted; logs alone may not preserve the visual symptom.

14. Build, dependencies and source handoff

The source archive contains VROSTBridge's own host/client code, tests, assets, scripts, twenty-three ALVR patches, notices and selected native Linux payloads. It is not an offline mirror of every upstream dependency and not a complete Windows installer (host or client). The build scripts fetch pinned upstream sources and dependencies. Internet access and platform toolchains are required for clean native builds. From 1.0.0.11 each release also offers the source split by part (the rendering PC, the Windows client and the Bazzite client), each archive holding what its part is built from with a `SOURCE.md`, from the same file list as the whole source (`scripts/package_component_sources.py`; every file belongs to a part or is named as whole-source only).

Component	Pinned baseline / role
ALVR	`f5023f8e78fa14ea57fb1038548d7820289a8524`; host and client-core transport
OpenVR	`0924064316de3effbcd1acf1e309182a2deb1c05`; headers/loaders
Rust	1.97.1 toolchain; native adapter/host build
Linux FFmpeg	n6.1.4 / `34277e12e80031c7f89494ba543684bc1dd0be8f`; minimal shared decode build (H.264, HEVC, AV1 with VA-API)
Windows FFmpeg (1.0.0.7)	The same n6.1.4 source snapshot (SHA-256 checked); minimal shared decode build (H.264, HEVC, AV1 with D3D11VA, the Vulkan hardware context, Windows threads), cross-compiled with mingw-w64 GCC 13 (win32 thread model), libgcc linked statically; arguments in `third_party/ffmpeg/BUILD-Windows.txt`
Windows client toolchain (1.0.0.7)	LLVM 19 or later (clang-cl, lld-link, llvm-lib, llvm-rc, llvm-mt) with a licensed Windows SDK and CRT in xwin layout, CMake 3.24+ and Ninja; Rust 1.97.1 for `x86_64-pc-windows-msvc`; NSIS 3.09; the host payload's embedded CPython 3.13.12; Wine 9.0 with Mesa's lavapipe for the tests
go-bsb-cams	`d0df45de2d9ce1a7f7febf6a2c1313812e5df287`; separate camera helper
Baballonia	Not shipped. RC4.5 downloads release `v1.1.1.0rc6` (`Baballonia.x64.v1.1.1.0rc6.tar.xz`, 517,439,412 bytes, SHA-256 `06c9ce7ecca19a0fee3ead40ef0c705ca173916e11a3315b99df63d348ea906a`, source `7339b07`) on the user's request; integration audit also against `5dbd332bc7f94868ec9b11a1b9ebd3c28acb91be`. The Windows client (1.0.0.7) downloads the same release's installer, `Baballonia.Setup.v1.1.1.0rc6.exe` (423,324,292 bytes, SHA-256 `dec2e8fd1e5c844662f99b47680d772a393f6eb2fdf4e4dd3c62853928b2c470`), and checks three of its installed program files by SHA-256

The ALVR patches cover microphone consent, network resilience, native controller fidelity, host capture/AMF, native protocol/build behaviour, idempotent chaperone application, the host frame-timing lock, client stream statistics and loss policy, host stream options and pacing, gaze-driven and optical-centre foveated encoding, AMF intra refresh, AMF output drain, the host prediction switch, the startup display configuration, the filtered reduced area (0016), the gaze hold (0017), exact colour and frames without a head pose counted (0018), the gaze comfort zone (0019), and the plain-SDR colour variant with the gaze-reach and still-controller diagnostics (0020). `scripts/fetch_upstream.py` checks revision identity and applies the ordered patch set. Do not substitute arbitrary current upstream builds or an unrelated ALVR dashboard; native protocol and build-manifest checks intentionally reject incompatible payloads.

Windows host builds use Windows x64 with Visual Studio 2022 C++ tools/SDK, Git, Python, CMake and the pinned Rust toolchain, or the Linux cross-build `scripts/Build-Host-Cross.py` (Clang/LLD MSVC ABI with a licensed Windows SDK) used for RC3 to RC4.1, 1.0.0.1 and 1.0.0.3 to 1.0.0.6; RC4.2 to 1.0.0 reused the RC4.1 driver and 1.0.0.2 the 1.0.0.1 driver. From 1.0.0.4 `scripts/host_module_tests.py` builds the driver's own VROSTBridge Rust modules (gaze foveation, stream options, pose drops and, from 1.0.0.6, the controller pair check) with stand-ins for the ALVR types they use and runs their tests on Linux. Release drivers are built without ALVR's `gpl` feature. Linux builds use C++17, CMake, Rust, FFmpeg, Vulkan, VAAPI, libdrm, Wayland, SDL2 and audio build prerequisites. Build in a development environment, then run the physical client in the actual Bazzite desktop/Game Mode session. The companion packaging freezes Linux Python and embeds Windows Python; source launch and end-user install are separate paths.

The Windows client (1.0.0.7) is cross-built on Linux too, in four steps: `scripts/Build-FFmpeg-Windows.sh` (FFmpeg's DLLs from the shipped source, checked by SHA-256, refusing a result that is not LGPL or that names a build folder), `scripts/Build-Client-Transport-Windows.py` (`vrostbridge_transport.dll` for `x86_64-pc-windows-msvc` with a static C runtime, checked for every `bridge.h` export, for runtime DLLs Windows does not provide and for build paths), `scripts/Build-Client-Windows.py` (import libraries, then CMake with Ninja and `scripts/cross/windows-clang-cl.cmake`, the C runtime linked statically, installed into `dist/client-windows` and checked for build paths; `--test` runs the C++ contract tests under Wine, the Vulkan ones on lavapipe) and `packaging/build_windows_client.py` with the launcher of `packaging/build_windows_launcher.py --role client`. The package builder refuses native programs that do not carry the release's version, a file that is not x64, a program that imports a Visual C++ or MinGW runtime DLL, FFmpeg DLLs not built from the source the package carries, and an embedded Python that differs from the verified host payload's; it gives every file one time and mode, so the same inputs give the same packages. `scripts/Build-Windows.cmd` builds and runs the contract tests natively with Visual Studio's CMake (without the Windows client). See `BUILDING.md`.

See `BUILDING.md` and `../packaging/README.md` for commands, output locations and release assembly constraints. Host packaging can reuse a verified host payload only after hashes/provenance and the current native patch manifest pass. Pinned inputs and hash checks aid repeatability; byte-for-byte reproducible builds across arbitrary systems are not claimed.

15. Attribution and publication readiness

Retain `THIRD_PARTY.md`, `OPEN_SOURCE_CREDITS.md`, licence files, vendored notices and dependency source/build records. ALVR is reused under MIT, OpenVR under BSD-3-Clause, the minimal Linux FFmpeg build under LGPL-2.1-or-later, and the camera helper under GPL-2.0. From 1.0.0.7 the Windows client's minimal FFmpeg build is LGPL-2.1-or-later too, built with mingw-w64: its package carries the unmodified source and the Windows build notes beside the DLLs, as the Bazzite package does, and the mingw-w64 runtime notice for the startup and support code linked into them (libgcc, also linked in, is under the GCC Runtime Library Exception, which asks for no notice). These are distinct components and distribution obligations; a credits page alone does not satisfy every obligation.

Hosts RC1-RC3 carried the upstream ALVR host's separate GPL-3.0-or-later FFmpeg build (`n8.1.2-54-gc573a95381-20260917`). From RC4 the driver is built without ALVR's `gpl` feature, imports no FFmpeg, and the host package no longer contains it, so that source/build closure no longer applies to new releases. RC1-RC3 host packages must never be republished.

From 1.0.0 VROSTBridge's own code is under the MIT licence (`LICENSE`, copyright JagerBombastic), and the public-release review is done: `NOTICE` carries the key attributions, including NVIDIA's required statement for the Video Codec SDK sample code compiled into the Windows driver; `THIRD_PARTY.md` lists every shipped component with its licence file (AMD AMF, nvEncodeAPI and backward-cpp MIT notices, the ALVR Dashboard's fonts, the libraries inside the frozen Linux companion, the Apache-2.0 part of websockets, the DRM lease protocol, the MPL-2.0 crates and where their source is); both packages carry `LICENSE`, `NOTICE`, `THIRD_PARTY.md` and `licenses/`; the camera helper's changes are listed with its source,

and its only dependency without a stated licence (the MJPEG module) is replaced by VROSTBridge's own code. Baballonia and VB-CABLE stay downloads from their authors at the user's request, under their own licences. The Windows client package (1.0.0.7) carries the same `LICENSE`, `NOTICE`, `THIRD_PARTY.md`, `licenses/` and `docs/`, plus FFmpeg's source and Windows build notes and OpenVR's licence under `third_party/`. Test data uses made-up LAN addresses and placeholder serials. `docs/PUBLISHING.md` is the checklist for a public GitHub repository (a fresh history, secret scan, release assets).

This is an engineering review, not legal advice. VROSTBridge is an independent open-source project. It is not affiliated with, sponsored by, or endorsed by Valve Corporation, Bigscreen, or any other headset, tracking-system or platform manufacturer, nor by the ALVR project or Project Babble. Third-party product names and trademarks belong to their respective owners; references to them identify compatibility or dependencies and do not imply endorsement or certification. The logo and icons are the project's own artwork (brand pack in `brand/`), licensed CC BY-ND 4.0 from 1.0.0.11 (`brand/LICENSE.md`: shared unchanged with credit; forks use their own name and logo); no trademark registration or clearance is claimed.

16. Validation and release acceptance

16.1 Automated gates

Preserve native contract tests for frame selection, geometry/IPD, projection round trips, skeletal/input handling, readiness, chaperone and DRM-lease protocol. Preserve Python tests for profile validation, lifecycle ownership, network/pairing policy, reports, install maintenance and UI settings. Check packaged executable startup, frozen resources, authenticated local HTTP, staged manifest integrity and archive CRC. Upgrade/repair tests must prove user state survives and only obsolete owned files are removed. The driver's VROSTBridge Rust modules have their own tests (`scripts/host_module_tests.py`, 1.0.0.4; with the pose-drop counter from 1.0.0.5), and the driver's colour rules theirs (`host_color_tests`, 1.0.0.5). The camera helper's Go tests run under the race detector (`CGO_ENABLED=1 go test -mod=vendor -vet=off -race .`).

From 1.0.0.7 the Windows client has its own gates, all run on Linux: the Python tests of its Windows modules with stand-ins for the Windows calls (`tests/test_windows_network.py`, `test_windows_power.py`, `test_windows_runtime.py`, `test_windows_eye_tracking.py`, `test_windows_ui.py`, `test_windows_packaging.py` and `test_vrostbridge_1007.py`, which also checks that every package carries one version); the C++ contract tests cross-built and run under Wine 9.0 with Mesa's lavapipe through winevulkan (`scripts/Build-Client-Windows.py --test`: the shaders (until 1.0.0.10 both fused-pass variants), the plane upload, CPU decoding with the shipped FFmpeg DLLs, gaze over loopback UDP, and the command pipe and console events); the page checks with the Windows wording; the package builder's own checks; and Wine runs of the installer and uninstaller and of the companion with its page. Wine has no D3D11 video decoder and lavapipe gives no GPU LUID, so the D3D11VA path, and the real Windows Firewall, Wi-Fi, power and Baballonia installer calls, are reviewed and built but not run: they are hardware gates (section 16.2).

From 1.0.0.11 `tests/test_vrostbridge_1011.py` covers the version, the frame pass's source (one shader, its constants and spans, the dispatch per eye, the clears), the lens area's sources, the GPU timing fields, the `scale` command, the output scale's rules with TIMING seconds and in the companion, the timeline and the pages; `tests/fused_shader_tests.cpp` compares the pass with its CPU model and the per-frame constants with ALVR's formula, `tests/visible_area_tests.cpp` the spans, `tests/shown_size_tests.cpp` the scaled size, and `tests/video_pass_tests.cpp` runs the client's own frame path (`VulkanVideo`) from decoded planes to the image SteamVR is given, with a stand-in SteamVR: on Linux from planes filled as the VA-API copy fills them (the VA-API import itself is not run), under Wine from CPU-decoded frames through the Windows client's upload.

From 1.0.0.10 `tests/test_vrostbridge_1010.py` covers the version, the menu button's handling in the native client's source (its press, release and focus rules are contract-tested in `tests/input_tests.cpp`),

the lease adapter's socket name and client lines in its source, the lease runtime against a stand-in adapter, SteamVR's environment in Game Mode, the early stop on the compositor's lines, the kept Game Mode start, the arrival check, and the base stations' three tries and signal against a stand-in BlueZ.

From 1.0.0.9 `tests/test_vrostbridge_1009.py` covers the version, the patch series and the driver's new options, the parity settings per driver, the host's received reports, the base stations' retry, Baballonia's CPU weight, the game-audio status and timeline, and the transport's interface; the patched ALVR crates' own tests (`cargo test -p alvr_sockets -p alvr_audio`) cover packet repair, the player against simulated clocks, bursts, stalls and losses, and the capture's conversion and downmix.

From 1.0.0.8 `tests/test_vrostbridge_1008.py` covers the bitrate ceiling and governor (including the decoding check), the pacing headroom, the host's live bitrate change against a stand-in for ALVR's interface, the performance balance against a stand-in steamos-manager, the base stations against a stand-in BlueZ and the PowerShell script's parse, gaze centring through the real relay, the calibrations, and the pages and Game Mode rows; `tests/test_desktop_ui.js` checks the page's new lines.

Documentation refresh verification checks internal links, source/runtime preservation and archive hashes; it does not reclassify earlier hardware tests as newly run. A full release needs current native and Python suites plus real install/run gates on each supported OS.

16.2 Physical acceptance matrix

Test	Required observation
Index desktop, Ethernet and Wi-Fi	Correct stereo, stable head motion, actual refresh/stream size visible, audio intact
Beyond desktop at local working mode	RC1.3 optical centre/FOV correct, stable scale and alignment, no headset-ready timeout
Controller soak, at least 30 minutes	Both hands retain actions and skeletons; brief headset occlusion keeps held inputs; sleep/reconnect recovers; host menu and haptics (full length, stop) work
Frame-timing lock (driver with patch 0007)	Quality line reports the lock steering; frames wait about 3-4 ms for the headset with small spread; no repeat/drop judder across repeated Stop/Start
Foveated encoding (RC4)	No visible seam or edge blur at Subtle on Index; Beyond Eye-tracked looks sharp wherever the user looks, without lag or shimmer; encoded size reported
Network test, pacing and recovery (RC4)	Test result plausible on Ethernet and Wi-Fi; pacing reduces losses on a slower client link; gradual recovery confirmed and losses heal without freezes
Power and wake (RC4)	Host stays awake in long sessions, wakes by Wake-on-LAN from sleep, sleeps after the configured time; client keep-awake and headset audio restore afterwards
Steady picture (RC4.2)	Head-locked text and held objects stay still while turning; alignment check answered; `pose_slot_changes` small
Wi-Fi without stalls (RC4.2)	No periodic stall over 15+ minutes on Wi-Fi; Wi-Fi works normally after Stop; Ethernet detected with its speed
Host microphone (RC4.3)	Host card says Ready with VB-CABLE installed; Set up headset microphone on a PC without it downloads, verifies and opens VB-CABLE's setup; the headset microphone works in Discord
Graphics limit (RC4.3)	Standard holds the headset refresh; at High on a weaker headset PC the in-headset line and page warning appear, re-aim pauses and Use Standard applies at the next Start
GPU clock (RC4.4)	With the TDP limit raised, any GPU clock setting on automatic and the charger connected, the GPU readout climbs well above 800 MHz, High holds 75 fps and tracking is steady; at a low TDP the warning names the GPU clock and the power limit
Compositor crash (RC4.4)	If SteamVR's compositor on the client crashes, both pages and Game Mode say so and the session restarts once by itself
Headset text (RC4.4)	The quality pill and status cards look crisp
Motion prediction on/off	Lower hand/controller lag and fewer view-edge artefacts with prediction on, without visible overshoot; frame age reported from sampling time
Repeated cold/warm Start/Stop	Both runtimes start/stop cleanly; first launch is not consistently worse than second
IPD while streaming	Eye geometry changes without discontinuity or mismatched source-frame reprojection

Test	Required observation
Room-scale setup	Host standing origin/rectangle agrees; local physical boundary remains correct
Loss and recovery	Brief loss holds remote view; longer loss is neutral; stop/retry remains available
Game Mode	Built-in screen stays on with the HMD connected; Steam entry and artwork; Beyond 2e recognised and reserved; HMD lease/display, every setting, B never exits, startup, stopping and return to desktop all work
Beyond 2e optional gaze	Correct calibrated combined direction, refit/restart behaviour and stale-data rejection
Managed eye tracking (RC4.5, 1.0.0)	Set up eye tracking downloads and verifies Baballonia; a Desktop Mode session runs the eye cameras and Baballonia (about 30 samples/s) with both eye pictures in their areas; camera access granted without reconnecting if asked; the Eye calibration shown in the headset over the stream during a session, each eye correct, with the session still running after training, then the region follows the eyes in Desktop and Game Mode with no Baballonia window there; closing and reopening Baballonia, and the CPU cost at the user's TDP
Gaze hold (1.0.0.4)	Fine detail near the edge stays sharp while looking at it and blinking; the host's `VROST_GAZE_FOVEATION` shows `held_frames` growing and `fallback_frames` far below `gaze_frames`
Processor and power limit (1.0.0.4)	With the GPU clock fixed at its maximum under a shared power limit, the warning appears within about 15 s of streaming; Set the GPU clock to automatic clears it and the eye-tracking sample rate rises
Wired pacing (1.0.0.4)	A path tested before 1.0.0.4 is tested once more; the Stream: line names the loss-free pace; lost frames far fewer than in the 1.0.0.3 test (146 in 8 minutes)
Short stalls (1.0.0.4)	A stall under a second keeps the last frame instead of turning grey
Game Mode controls (1.0.0.4)	The D-pad and stick step through rows, left and right change values, a held direction repeats, one press moves once
Exact colour (1.0.0.5)	With 10-bit colour on, black is as black on the OLED panels as on the PC; the host's `VROST_COLOR` line says `bits=10 range=full … verified=1`, or the client page says why 8-bit was used
Eye cameras (1.0.0.5)	Eye-tracking samples a second close to the camera pictures a second, steadily; the STATS lines show few lost pictures and no re-openings; the camera helper's processor time far below a core
Comfort zone (1.0.0.5)	The sharp region's edge stays still while the eyes wander a little and follows at once a glance across the view
Headset not tracked (1.0.0.5)	With the base stations blocked, Headset not tracked within a few seconds and the picture back on its own; the host's `VROST_NO_HEAD_POSE` counts the dropped frames
Network test (1.0.0.5)	A path whose last test found no clean rate is tested again before the next session; on wired Ethernet the bitrate is well above 30 Mbit/s
Gaze reach (1.0.0.6)	After looking to the far left, right, up and down, Gaze reach is close to how far you looked (up to 45°); after calibrating again with wider head turns the sharp region follows toward the edges

Test	Required observation
10-bit colour (1.0.0.6)	`VROST_COLOR_PROBE` shows the plain SDR attempt; with `variant=plain verified=1` the video details say 10-bit full range and blacks stay black
Network test (1.0.0.6)	The first session tests again; on Ethernet the bitrate is well above 50 Mbit/s, with `network_spread_p95_ms` small at every clean step
Controllers (1.0.0.6)	Touched together and held still for five seconds, the client's `pair` distance, the host's `received_mm` and `submitted_mm` agree and `max_shift_mm` is under a millimetre; tapping shows the overshoot in the prediction check's `max_mm`
Ultra (1.0.0.5)	The video details say shown at SteamVR's size, and the frame rate holds better than in the 1.0.0.4 test
Windows client install (1.0.0.7)	The setup runs without administrator rights (after SmartScreen's More info → Run anyway) and the page shows VROSTBridge Client 1.0.0.7; host and client on one PC each uninstall alone; the uninstaller's firewall prompt (approved and declined) and its Baballonia question; the native programs and DLLs load on a PC without the Visual C++ runtime
Windows client session (1.0.0.7)	Allow home-network connection (one UAC prompt) and the host is found; network test and Start; for several minutes the frame rate and lost frames hold, head and controllers are right, the video details name D3D11VA and the copy time (`map_sync_import`) stays small; Stop closes SteamVR on both PCs
Windows tray and sign-in (1.0.0.7)	Stop and Start from the tray icon, with the notifications for streaming, a stopped session and one that did not start; Start when I sign in starts the tray only
Windows power and Wi-Fi (1.0.0.7)	Best performance during a session and the user's own power mode after Stop (and after a crash, at the next start); Wi-Fi behaves as before after Stop; on Windows 11 24H2 and later, the location note when permission is missing
Windows eye tracking (1.0.0.7)	With Bigscreen's eye-tracking app closed, Set up downloads, checks and installs Baballonia with no Start menu entries of its own; the `BigEye` camera gives samples; Show Baballonia and the Eye calibration show over the stream; the sharp region follows the eyes and Gaze reach is plausible
SteamVR not set up (1.0.0.7)	On a Windows PC where SteamVR has never run, Start says so in plain words
Frame pass on any driver (1.0.0.7, one shader from 1.0.0.11)	On a driver that offers only 128 bytes of push constants the client uses the fused pass, and the picture, its sharpening and the re-aim are right
Steadier streams (1.0.0.8)	With Use recommended settings, the Stream: line names the picture ceiling (49 Mbit/s for a Beyond 2e at Standard, eye-tracked, 75 Hz); decode time near 1.0.0.5's; the timing lock steering; lost frames rare; any `BITRATE` step has a plausible reason and the host's `VROST_BITRATE` follows it
Performance balance (1.0.0.8)	On the charger the power limit rises at the start of streaming; the GPU clock leaves 800 MHz when frames run late; eye tracking keeps its rate; Steam's Performance menu shows the user's own settings after Stop and after a crash and restart
Base stations (1.0.0.8 to 1.0.0.10)	Closing VROSTBridge puts every 2.0 base station to sleep within about 10 s on Bazzite and Windows; Start wakes them all and the headset is tracked sooner than in the 1.0.0.7 test; the card names the stations and the last result, on Bazzite with the signal of a station not reached

Test	Required observation
Game audio (1.0.0.9)	Several minutes of music or dialogue without crackle, lag or breakups; the Game audio line at no dropouts; the host's `VROST_AUDIO_CAPTURE` line without `fallback=cpal`
Menu button (1.0.0.10)	Short presses open the rendering PC's SteamVR menu every time and long ones recentre; one `SYSTEM_BUTTON` line per press, `unread=0`; a trigger or grip held through a short press stays held
Game Mode handover (1.0.0.10)	Start in Game Mode streams to the headset; if it does not, the client report's `game_mode_last_start` shows the display SteamVR's server was given, the `LEASE_CLIENT` lines and the compositor's own lines
Bitrate on Wi-Fi (1.0.0.10)	With Wi-Fi power saving off and a new network test, the bitrate holds; a step for uneven arrival alone is undone within about 20 s when frames arrive no more evenly
Bitrate after a slow start (1.0.0.12)	When the GPU clock is held higher, the bitrate goes back to where the session started (`BITRATE … (back to the start: the GPU clock was raised)`); seconds in which the rendering PC sent nothing no longer step it down
GPU clock from the start (1.0.0.12)	The second session on the same PC says GPU held at … MHz from the start; Steam's Performance menu shows the user's own setting after Stop
Add-on warning (1.0.0.12)	The host page names only add-ons installed from outside SteamVR, or nothing
Native on the headset PC (1.0.0.11)	At Native with eye-tracked foveation the stream holds the headset's refresh; `frame_pass_gpu` and `plane_copy_gpu` are recorded with SteamVR's `gpu_ms` and `compositor_gpu_ms`, at Native and at Standard
Lens area (1.0.0.11)	Nothing at the edges of the view looks cut off or dark, also while turning the head quickly; `VISIBLE_AREA` gives each eye's share; the picture is the same with Skip what the lenses never show off
Keeping the frame rate (1.0.0.11)	Where the headset PC cannot hold the refresh, the page shows Shown at 90 % (or 80 %) instead of half rate, with a `SCALE` line, and full size returns when there is room; off, the size stays at 100 %
Packet repair (1.0.0.9)	At Native (until 1.0.0.10 High) with eye-tracked foveation, Rebuilt on the way counts frames while lost frames stay rare and the bitrate holds
Gaze centring (1.0.0.8)	After a minute the card's resting gaze is near straight ahead for a good calibration; for an off-centre one the sharp region moves to where the user looks; Use previous calibration and Reset centring do what they say
Install/repair/uninstall	Fresh and upgrade paths work without destroying preferences or unrelated software

Run a controlled performance comparison at the same game scene, resolution, codec, bitrate and refresh. Record host and client hardware, route, runtime/driver versions and reports. Compare fresh/unique frame cadence and p95/p99-like summaries only where the sampling supports them. Set numeric release targets from this repeatable baseline; there is no demonstrated universal latency promise in the current evidence.

17. Priorities and future work

1. Confirm 1.0.0.12 on the reference setup (section 16.2): the bitrate going back up once the GPU clock is held, the clock held from the next session's start and the add-on warning, with reports from both PCs; and find why the rendering PC's frames reached the capture late while a game ran (the 1.0.0.11 test's stalls): a session with eye tracking off and one with another game tell the game's share from the gaze's.

1. Of 1.0.0.11 (its frame pass ran at 1.9 ms of GPU time a frame at Native, and on a Beyond the lens area keeps 99.7 % of the pixels): showing smaller where the graphics fall behind, and the lens area on other headsets; then whether sampling the decoder's surfaces without the plane copy is worth its costs.

1. Confirm 1.0.0.10's menu button, Game Mode handover, bitrate on Wi-Fi and base stations on the reference setup (section 16.2), with reports from both PCs (1.0.0.9's game audio and packet repair worked in its test); then 1.0.0.8's performance balance and gaze centring, and tune the governor's thresholds from the reports if needed.

1. Run the Windows client (1.0.0.7) on Windows hardware for the first time: install, firewall prompt, network test and a session of several minutes with D3D11VA decoding and its copy time, the tray and sign-in start, the power mode and Wi-Fi put back after Stop, and eye tracking with Baballonia's Windows installer and its calibration over the stream (section 16.2), with reports from both PCs. Then physically test 1.0.0.6's changes, which 1.0.0.7 carries unchanged: Gaze reach before and after calibrating with wider head turns, the sharp region toward the edges, the 10-bit attempts (`VROST_COLOR_PROBE`), the network test repeated and its bitrate, and the controllers held still and tapped together (`controller_prediction`). Then 1.0.0.5's open checks: black with 10-bit colour on and the `VROST_COLOR` line, the network test repeated and the bitrate it sets, eye-tracking samples and camera pictures a second, the sharp region still for small eye movements, Headset not tracked, and Ultra shown at SteamVR's size; then 1.0.0.4: the network test run once more and fewer lost frames, the processor warning and Set the GPU clock to automatic, the sharp region through blinks and pauses, Beat Saber without grey flashes, Game Mode's D-pad and stick, and High (2560) with eye-tracked foveation once the processor keeps its clock; then 1.0.0.3: the Game Mode session past the display reservation, Stop closing the host's SteamVR after a slow start, the reduced area (steadier, softer) and quick eye movements with eye-tracked foveation, the eye-tracking view, and an update that keeps Baballonia; then 1.0.0.2's Baballonia Eye calibration in the headset over the stream on Bazzite (confirmed working in Desktop Mode in the 1.0.0.2 test), then a fresh installation on both PCs with its names and icons (Start menu, tray on light and dark taskbars, sign-in startup, Bazzite menu and pin, Add to Steam, pairing) and a whole session with the rebuilt driver, then RC4.6's and RC4.5's eye tracking (set up eye tracking, the first Desktop Mode session with the eye cameras and Baballonia, camera access if asked, the Eye calibration in the headset during the session, the region following the eyes in Desktop and Game Mode, closing and reopening Baballonia, fps and GPU readout with eye tracking on and off) and RC4.4 (raise the TDP limit first, any GPU clock setting on automatic, and watch the GPU readout: High at 75 fps with steady tracking, the power-limit warning's MHz if not, crisp headset text, eye-tracked settings and note, compositor crash recovery), RC4.3 (host microphone card, Game Mode start with the Beyond 2e and its settings and B button, frame rate at Standard and the graphics notice at High), RC4.2 (steady head-locked content and the alignment check, Wi-Fi without scan stalls, Ethernet detection, Game Mode screen, Steam entry and built-in display, in-headset cards, report upload), RC4.1 (single-step prediction, encoder backlog, recovery after a Wi-Fi dropout, startup time) and the RC4 features (foveated encoding on Index and Beyond 2e with eye tracking, 10-bit, network test and pacing, gradual recovery, in-headset status, wake/sleep, quick resume) together with the RC3/RC2 items, and confirm the RC1.3 Beyond projection correction; check Index for regression.

2. Confirm the frame-timing lock removes phase-dependent waiting and judder on hardware; use the cadence class for any remaining half-rate periods.

3. Complete controller soak/reconnect, first-launch consistency and repeated lifecycle acceptance.
4. Validate Game Mode on the real Bazzite session and the complete Beyond 2e calibrated gaze path.
5. Close public-release licensing/source and Windows installer qualification items (from 1.0.0.7 both Windows installers, and the notices of the Rust crates only the Windows client's transport uses, listed in `THIRD_PARTY.md`); then decide whether the evidence supports an alpha or a stable release label.
6. Evaluate additional pacing/congestion ideas against a measured baseline. Apollo/Sunshine-style remote-desktop techniques are research references, not a shipped replacement or proven drop-in VR upgrade. Any replacement must preserve stereo optics, pose timing, controller/skeleton data, menus and compositor integration.

Decoding on the Windows client without the copy through system memory (the decoder's Direct3D 11 texture shared with Vulkan, or Vulkan Video decoding), code-signed Windows installers, Internet transport, HDR and dynamic foveated rendering for games remain separately scoped future projects. Keep working Index behaviour as the regression baseline while extending Beyond support.